

中图分类号:

UDC:

学校代码: 10055

密级: 公开

南开大学
硕士学位论文

GPU 近似最近邻检索系统架构设计

Architecture Design of a GPU-based Approximate Nearest
Neighbor

论文作者	陈源辉	指导教师	刘晓光教授 教授
申请学位	硕士	培养单位	南开大学
学科专业	计算机技术	研究方向	向量检索系统
答辩委员会主席		评阅人	

南开大学研究生院

二〇二四年五月

南开大学学位论文使用授权书

本人完全了解《南开大学关于研究生学位论文收藏和利用管理办法》关于南开大学(简称“学校”)研究生学位论文收藏和利用的管理规定,同意向南开大学提交本人的学位论文电子版及相应的纸质本。

本人了解南开大学拥有在《中华人民共和国著作权法》规定范围内的学位论文使用权,同意在以下几方面向学校授权。即:

- 1. 学校将学位论文编入《南开大学博硕士学位论文全文数据库》,并作为资料在学校图书馆等场所提供阅览,在校园网上提供论文目录检索、文摘及前 16 页的浏览等信息服务;
- 2. 学校可以采用影印、缩印或其他复制手段保存学位论文;学校根据规定向教育部指定的收藏和存档单位提交学位论文;
- 3. 非公开学位论文在解密后的使用权同公开论文。

本人承诺:本人的学位论文是在南开大学学习期间创作完成的作品,并已通过论文答辩;提交的学位论文电子版与纸质本论文的内容一致,如因不同造成不良后果由本人自负。

本人签署本授权书一份(此授权书为论文中一页),交图书馆留存。

学位论文作者暨授权人(亲笔)签字: _____

20 年 月 日

南开大学研究生学位论文作者信息

论 文 题 目	GPU 近似最近邻检索系统架构设计				
姓 名	陈源辉	学号	2120230659	答辩日期	
论 文 类 别	博士 ☐ 学历硕士 ☐ 专业学位硕士 ☒ 同等学力硕士 ☐ 划 ☒ 选择				
学院(单位)	计算机学院		学科/专业(专业学位) 名称		计算机技术
联 系 电 话	18359715052		电子邮箱	yhchen@nbjl.nankai.edu.cn	
通讯地址(邮编): 300000					
非公开论文编号			备注		

注:本授权书适用我校授予的所有博士、硕士的学位论文。如已批准为非公开学位论文,须向图书馆提供批准通过的《南开大学研究生申请非公开学位论文审批表》复印件和“非公开学位论文标注说明”页原件。

南开大学学位论文原创性声明

本人郑重声明：所呈交的学位论文，是本人在导师指导下进行研究工作所取得的研究成果。除文中已经注明引用的内容外，本学位论文的研究成果不包含任何他人创作的、已公开发表或者没有公开发表的作品的内容。对本论文所涉及的研究工作做出贡献的其他个人和集体，均已在文中以明确方式标明。本学位论文原创性声明的法律 responsibility 由本人承担。

学位论文作者签名：_____ 年 月 日

非公开学位论文标注说明

(本页表中填写内容须打印)

根据南开大学有关规定，非公开学位论文须经指导教师同意、作者本人申请和相关部门批准方能标注。未经批准的均为公开学位论文，公开学位论文本说明为空白。

论文题目			
申请密级	☐ 限制 (≤2 年) ☐ 秘密 (≤10 年) ☐ 机密 (≤20 年)		
保密期限	20 年 月 日至 20 年 月 日		
审批表编号		批准日期	20 年 月 日

南开大学学位评定委员会办公室盖章 (有效)

注：限制 ★2 年 (可少于 2 年); 秘密 ★10 年 (可少于 10 年); 机密 ★20 年 (可少于 20 年)

摘要

在大语言模型（LLM）与检索增强生成（RAG）技术爆发式增长的背景下，近似最近邻检索（ANNS）已成为现代人工智能基础设施与搜索推荐引擎的核心基石。图形处理器（GPU）凭借其海量并发计算能力，在加速 ANNS 领域备受关注。尽管现有研究显著提升了大规模检索的吞吐量，但在面向在线服务的低延迟场景中仍面临严峻挑战。

在调度层面，由于 ANNS 的近邻图检索具有显著的长尾特性，在传统的 GPU 阻塞式批处理模型中，处理较快的查询被迫等待长尾查询，导致批次间出现严重的调度气泡与资源闲置。此外，频繁的核函数启动与跨设备通信开销进一步加剧了系统的端到端延迟。在算法执行层面，GPU 的并行架构极大加速了距离计算开销，使得近邻图检索中关于候选队列与相关数据结构维护的开销占比突出，这类维护操作存在一定程度串行，限制了性能的进一步提升；目前低延迟的 Multi-CTA 并行模式下引入了的各 CTA 分片结果合并的开销，这需要跨 SM 同步与全局显存的参与，抵消了并行计算带来的部分收益。

针对上述挑战，本文设计并实现了一种面向低延迟场景的高性能 GPU 向量检索系统，分别从请求调度与算法执行两个层面进行了设计与优化。在调度架构层面，本文提出了一种基于多槽位的动态流式处理机制。该机制摒弃了传统的同步 GPU 批处理屏障，将计算资源抽象为逻辑独立的执行槽位，允许查询任务异步进入与退出，从而彻底消除了长尾查询对整体批次的阻塞效应。为配合动态调度，系统设计了持久化核函数策略，通过驻留 GPU 的核函数替代频繁的核函数启动，实现了任务的零开销分发；同时，引入基于本地副本的 PCIe 状态同步技术，利用统一地址空间技术（UVM）将 Host 与 Device 端的槽位状态映射到对方地址空间，有效消除了高频轮询下的 PCIe 总线 I/O 瓶颈。在算法执行层面，本文设计了软硬协同的计算优化策略。首先，提出分阶段束搜索算法，根据查询过程中的距离收敛特征，自适应地在“定位阶段”与“收敛阶段”切换搜索策略，通过延迟列表维护显著降低了维护列表的执行开销。其次，构建了 CPU-GPU 异构协同流水线，利用 GPU 专注搜索，CPU 异步合并的解耦设计，将逻辑复杂但计算量较小的 Top-K 归并任务卸载至 Host 端，在充分利用 CPU 空

闲算力的同时，有效掩盖了结果回传的延迟。

在基于 NVIDIA RTX A6000 硬件平台与 SIFT-1M、GIST-1M 等多个高维基准数据集的实验结果表明，本系统在保持高召回率的同时，显著突破了现有方案的性能瓶颈。与当前业界最先进的 GPU 图索引系统 CAGRA 相比，本系统在平均查询延迟上降低了 21.9% 至 35.4%，同时吞吐量提升了 27.8% 至 55.2%。消融实验进一步证实，调度架构与算法策略在不同负载下均表现出卓越的有效性，为构建兼具高吞吐与极速响应的下一代向量检索系统提供了有效的技术路径。

关键词： 向量检索；GPU；近似最近邻搜索；异构计算；低延迟系统

Abstract

Driven by the explosive growth of Large Language Models (LLMs) and Retrieval-Augmented Generation (RAG) technologies, Approximate Nearest Neighbor Search (ANNS) has become a fundamental cornerstone of modern AI infrastructure and search and recommendation engines. Graphics Processing Units (GPUs), with their massive concurrent computing capabilities, have garnered significant attention in the field of accelerating ANNS. Although existing research has significantly improved the throughput of large-scale retrieval, severe challenges remain in low-latency scenarios tailored for online services.

At the scheduling level, due to the significant long-tail characteristics of graph-based neighbor retrieval, traditional GPU synchronous batching mechanisms force faster queries to wait for long-tail queries in small-batch scenarios. This results in severe scheduling bubbles and resource idleness between batches. Furthermore, frequent kernel launches and cross-device communication overheads further exacerbate the system's end-to-end latency. At the algorithmic execution level, while the parallel architecture of GPUs greatly accelerates distance calculation, the overhead related to candidate queue and data structure maintenance in neighbor graph retrieval becomes prominent. These maintenance operations involve a degree of serial execution, limiting further performance improvements. Additionally, the current low-latency Multi-CTA parallel mode introduces overhead from merging the shard results of each CTA; this requires cross-SM synchronization and global memory participation, offsetting some of the benefits derived from parallel computing.

To address the aforementioned challenges, this paper designs and implements a high-performance GPU vector retrieval system tailored for low-latency scenarios, with optimization conducted at both the scheduling architecture and algorithmic execution levels.

In terms of scheduling architecture, this paper proposes a multi-slot-based dynamic streaming processing mechanism. This mechanism discards the traditional synchronous

GPU batching barrier and abstracts computing resources into logically independent execution slots, allowing query tasks to enter and exit asynchronously. This thoroughly eliminates the blocking effect of long-tail queries on the overall batch. To support dynamic scheduling, the system designs a persistent kernel strategy, replacing frequent kernel launches with a polling kernel that resides on the GPU, achieving zero-overhead task dispatch. Simultaneously, a PCIe state synchronization technology based on local copies is introduced. By utilizing Unified Virtual Memory (UVM) to map Host and Device slot states to each other's address space, this effectively eliminates the PCIe bus I/O bottleneck caused by high-frequency polling.

In terms of algorithmic execution, this paper designs a software-hardware synergistic computation optimization strategy. First, a Staged Beam Search algorithm is proposed. Based on distance convergence characteristics during the query process, the system adaptively switches search strategies between a "positioning stage" and a "convergence stage," significantly reducing the execution overhead of list maintenance through a lazy maintenance approach. Second, a CPU-GPU heterogeneous collaborative pipeline is constructed. Utilizing a decoupled design where the GPU focuses on searching and the CPU asynchronously merges, the system offloads the logically complex but computationally light Top-K merging tasks to the Host. This effectively masks result retrieval latency while fully utilizing idle CPU computational resources.

Experimental results based on the NVIDIA RTX A6000 hardware platform and multiple high-dimensional benchmark datasets, such as SIFT-1M and GIST-1M, demonstrate that the proposed system significantly breaks through the performance bottlenecks of existing solutions while maintaining high recall rates. Compared with CAGRA, the current state-of-the-art GPU graph index system in the industry, this system reduces average query latency by 21.9% to 35.4% and increases throughput by 27.8% to 55.2%. Ablation studies further confirm that the scheduling architecture and algorithmic strategies demonstrate excellent effectiveness under different loads, providing an effective technical path for building next-generation vector retrieval systems that combine high throughput with rapid response.

Key Words: vector retrieval; GPU; approximate nearest neighbor search; heterogeneous computing; low-latency systems

目录

摘要	I
Abstract	III
第一章 绪论	1
第一节 研究背景及挑战	1
第二节 本文主要工作	3
第三节 本文组织结构	4
第二章 背景介绍和相关工作	6
第一节 向量检索与近似最近邻搜索	6
第二节 ANNS 算法介绍	8
一、 基于近邻图的方法	8
二、 基于聚类与量化的方法	12
第三节 GPU 架构与 CUDA 计算模型	15
一、 硬件架构与执行模型	16
二、 层次化存储子系统	16
三、 CUDA 编程模型与软件抽象	17
第四节 GPU 加速 ANNS 的研究现状	17
一、 GPU 加速近邻图索引	17
二、 GPU 加速非图索引	21
三、 其他相关研究	22
第五节 本章小结	23
第三章 GPU 向量检索架构设计	24
第一节 研究动机	24
一、 请求调度层面的批处理气泡	25
二、 算法执行层面的性能瓶颈	26
第二节 系统总体架构	27
第三节 GPU 动态流式处理架构	28
一、 槽位状态管理	29

二、持久化 GPU 核函数	30
三、跨 PCIe 的状态副本	32
四、完整检索流程	33
第四节 并行检索算法优化设计	34
一、分阶段束搜索策略	34
二、CPU-GPU 异构协同流水线	37
三、Host 端多线程与 IO 优化	40
第五节 本章小节	41
第四章 实验设计与分析	42
第一节 实验环境	42
一、实验配置	42
二、实验数据集	43
第二节 综合性能对比实验	43
一、实验设置	43
二、不同召回率下综合性能比较	44
三、不同批次下综合性能比较	47
四、Top-K 召回规模对性能的影响	48
第三节 消融实验与结果分析	49
一、动态流式处理架构的有效性分析	49
二、多线程并发与状态副本机制的有效性分析	50
三、束搜索策略的有效性分析	51
四、本章小结	53
第五章 总结展望	55
第一节 本文工作总结	55
第二节 未来研究展望	56
参考文献	58
致谢	61
个人简历	62

第一章 绪论

第一节 研究背景及挑战

向量检索作为搜索引擎^[1-2]与推荐系统^[3-4]中召回的核心基石，近年来随着大语言模型（LLM）^[5]的发展与检索增强生成（RAG）^[6]架构的普及应用，其战略地位日益凸显。深入探究向量检索的架构优化与算法创新，不仅具有深远的学术价值，更对推动 AI 产业落地、构建高效智能的信息生态系统具有重要的现实意义。

从本质上看，向量检索代表了一种从“关键词匹配”向“内容理解”演进的信息检索范式转变。该模式依托深度神经网络驱动的嵌入模型，将非结构化物品信息映射为高维稠密向量（Embeddings），从而将抽象的内容理解转化为向量空间中的距离远近关系。在查询阶段，向量检索系统通过计算查询向量与物品向量之间的距离来召回最相关结果，有效跨越了传统检索中的对物品理解能力不足的问题。

在大规模数据集中，精确的 K 最近邻搜索需要计算查询向量与数据库中所有 N 个向量的距离，其计算复杂度为 $O(N \cdot D)$ （其中 D 为向量维度）。这种全库遍历的方式计算开销极高，难以满足在线服务对实时性的苛刻要求。为了在海量数据规模下降低检索时间开销，实际的向量检索系统广泛采用近似最近邻搜索（Approximate Nearest Neighbor Search, ANNS）技术^[7]，其核心理念在于剪枝需要计算距离的数据量，通过预先构建特定的索引结构将向量空间划分为若干区域或构建拓扑关系，使得查询过程仅需计算一小部分最相关的候选向量。这一机制本质上是在检索精度与时间开销之间进行权衡，即通过牺牲微小的精度损失换取查询性能的数量级提升，成为大规模向量检索落地的关键支撑。

当前的 ANNS 算法包括基于近邻图^[8-11]、基于量化^[12-15]、基于哈希^[7,16]和基于树^[17-18]的类型，其中基于近邻图和量化的方法占据了主流地位。具体而言，近邻图算法凭借其在召回率与检索性能之间优异的平衡能力，成为当前综合性能最强的技术路线；量化方法则以其极高的灵活性著称，能够与近邻图、倒排文件索引（IVF）等结构结合使用，在大幅压缩存储开销的同时保持高效的检索能

力。在工业界，以 Meta 开源的 Faiss 库^[19]，以及 Milvus、PostgreSQL (pgvector) 为代表的向量数据库^[20-21]已广泛集成了上述主流算法，极大地降低了高性能向量检索技术在实际生产系统中的集成与落地门槛。

不过在 AI 产业飞速发展的驱动下，向量检索系统需要处理的数据规模与向量维度的急剧膨胀，使各类 ANNS 算法在 CPU 计算模式下面临着严峻的性能挑战。与此同时，图形处理器 (GPU)^[22]凭借其拥有数千个计算核心的大规模并行架构，在处理运算与海量数据吞吐时展现出远超通用 CPU 的效率。为了突破性能天花板，学术界与工业界开始广泛探索利用 GPU 加速 ANNS 检索。GPU 已不再仅仅是 AI 模型的训练与推理载体，正逐步演变为向量检索系统高性能加速的关键硬件平台。

尽管在基于 GPU 的近邻图 ANNS 领域涌现了如 CAGRA 等高质量成果^[23-27]，特别是在吞吐量层面已经做到相比于 CPU 模式的指数级提升，但现有工作在从利用吞吐峰值性能走向在线生产环境的过程中，仍面临着请求调度层面与算法执行层面两方面的挑战。

在请求调度层面，为了充分释放硬件潜力，现有工作大多采取激进的大规模批处理策略^[23-26]，这在实际运用中需要等待大量的请求以供调度，无法满足在线搜索等业务对实时性的极高要求。即便是尝试通过小批量处理和两级并行架构来降低等待时间，但其本质上仍沿用了传统的阻塞式调度模型，本文在多个公开数据集中观察到，由于近邻图的搜索模式中每个查询的路径长度显著不同，具有明显的长尾延迟特性，即使是在较小的批次中也存在明显长尾。这导致在阻塞式调度模型下，批次内处理较快的查询被迫等待处理较慢的长尾查询。这种批次内长尾的现象会产生批处理间的气泡，使得 GPU 的并行计算优势利用不足，放大了整体的延迟。特别是在为了低延迟目标的小批量处理中，进一步放大的查询气泡的开销。

在算法执行层面，近邻图查询算法的核心通常基于贪婪搜索，其过程具有强烈的顺序依赖性，这与 GPU 擅长的规则化、大规模数据并行计算模式冲突。特别是随着 GPU 算力的提升，CPU 中占据主导的高维向量距离计算开销被极大压缩，这一变化让 GPU 近邻图查询中不规则的非计算密集型开销占比上升，成为新的瓶颈。此外，GPU 的硬件架构具有明显的资源局部性与层级化分组（类 NUMA 特性）。在处理对延迟敏感的查询时，为了提高资源利用率，需要采用大量计算单元的查询内并行策略，将单个查询映射到跨越多个物理分组的计算单

元上。这种跨越面临着高昂的同步屏障与通信延迟，限制了并行加速比的提升，这也是导致现有低延迟方案稀缺的关键技术瓶颈。因此，大多数现有研究妥协于高吞吐量目标，而牺牲了单次查询的响应速度。值得注意的是，近期 NVIDIA 提出的 CAGRA^[27] 通过引入 Multi-CTA 模式的双层并行架构，一定程度上缓解了上述问题。虽然该模式通过分片查询解决了小批量查询无法填满 GPU 核心的痛点，这类策略也引入了额外的局部分片结果合并开销，跨 SM 的结果聚合依旧需要全局显存的参与，抵消了并行计算带来的部分收益。总体而言，高效的 GPU 近邻图检索方案仍处于持续演进阶段，在细粒度并行化设计与性能优化上仍留有巨大的探索空间。

第二节 本文主要工作

针对上述挑战，本文面向低延迟场景设计了一种专用的 GPU 向量检索系统，并在请求调度与算法执行两个层面进行了优化。该系统通过以下设计更好地结合了近似最近邻搜索的检索特征与 GPU 的硬件特性，在延迟与吞吐之间取得了更优的平衡，相较于现有前沿的 GPU 向量检索系统展现出更强的综合竞争力。具体挑战和研究内容的对应关系如图1.1所示。

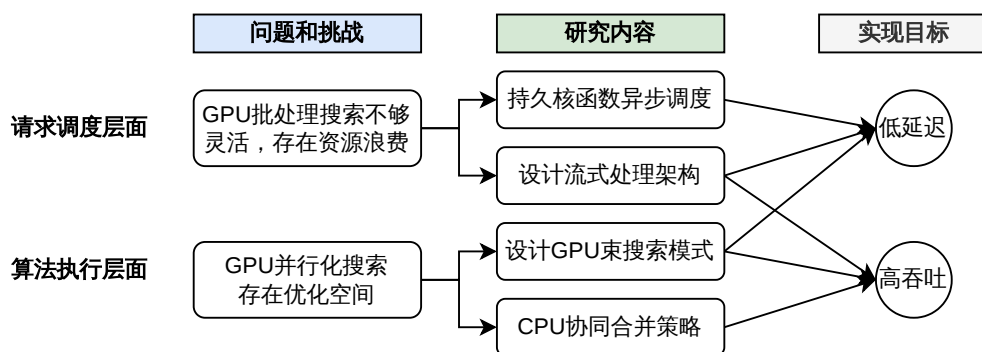


图 1.1 挑战和研究内容的对应关系

在请求调度层面：

- 针对近邻图检索中显著的长尾延迟特性，传统的阻塞式批处理模型面临严重的调度气泡与资源闲置问题。为此，本文打破了传统的批次同步屏障，设计了一种基于多槽位的 GPU 流式处理架构。该机制为每个逻辑槽位引入独立的状态管理，支持查询任务的异步进入与退出。这一设计彻底消除

了长尾查询对整体批次的阻塞效应，同时免去了为凑齐批次而产生的排队等待开销，实现了查询请求的即到即处理。

- 为了支撑细粒度的 GPU 流式处理机制并解决随之而来的调度瓶颈，本文重构了核函数调度模式。流式处理虽然实现了任务的即时响应，但其高频的任务分发导致了频繁的核函数启动，显著放大了元数据初始化等系统开销，并且传统的阻塞式核函数调用无法满足槽位灵敏的异步交互。为此，本文提出了一种持久化核函数策略，通过在 GPU 上驻留核函数，仅需单次启动即可支持后续连续的流式任务执行，彻底消除了重复启动开销。此外，结合读写副本机制优化了 CPU-GPU 跨 PCIe 通信，利用统一虚拟寻址（UVM）技术实现了状态的高效同步。

在算法执行层面：

- 基于对 GPU 近邻图查询的性能分析，本文发现维护图搜索候选中间结果的操作存在一定开销，限制了部分性能提升。为了降低维护操作占总开销的比例，本文提出了一种分阶段束搜索策略。该策略依据查询过程中的距离收敛特征，将搜索过程划分为“定位”与“收敛”等不同阶段，并自适应地降低非关键阶段的队列维护频率。实验表明，该方法在几乎不影响检索精度的前提下，大幅减少了冗余的排序与截断操作，有效降低了单次查询的延迟。
- 针对 Multi-CTA 两级并行架构中存在的部分结果合并问题，本文观察到 Top-K 合并操作虽然逻辑复杂但数据量较小，可以使用 CPU 处理。因此，本文提出了一种异构解耦设计，将其从 GPU 卸载至 CPU 执行。通过构建“GPU 专注并行搜索，CPU 异步处理合并”的异构流水线，充分利用了 CPU 的乱序执行能力，有效掩盖了数据回传与结果归并的延迟，提升了系统的资源利用率与综合吞吐量。

本文提出的 GPU 向量检索系统，提升实际搜索场景对低延迟需求，同时在吞吐量上依旧保持着相对优势，为现代搜索引擎引入 GPU 实时处理 ANNS 具有一定的借鉴和参考意义。

第三节 本文组织结构

本文一共分为五个章节，各章内容结构如下：

第一章绪论，介绍了本文的研究背景与期望解决的问题，提出了面向小批

量的低延时近邻图 GPU 向量检索系统，并分点列举了本系统的关键设计。最后介绍了全文的组织架构。

第二章将详细介绍本文的研究背景以及相关的研究工作。背景方面将贯穿 ANNS 的相关概念，主要的 ANNS 算法类型，GPU 硬件的相关架构介绍。然后延伸到基于 GPU 的 ANNS 相关研究上，并重点介绍近邻图算法类型的 GPU 工作，分析它们的优势和一些未解决的挑战。第三章，先介绍本工作的总体架构，然后详细阐述研究动机、各个模块的设计以及相关优化。首先，针对当前的小批量查询系统存在的气泡问题，提出 GPU 动态流式处理的架构，设计基于查询槽位的状态流转模式。配合流式调度实现低调用开销的持久化核函数，还通过副本减少跨 PCIe 数据传输。接着，根据当前的查询算法提出两方面的改进。包括将查询过程分阶段考虑，减少查询后期维护候选队列的次数；Multi-CTA 模式下将的合并操作卸载至 CPU，形成协同流水线。

第四章实验中，对本文提出的 ALGAS 系统进行了全面的实验评估与深入分析。使用多个数据集验证了 ALGAS 在相同的召回率水平下，相比于其他工作有着更高的吞吐量与更低的查询延迟。此外，通过消融实验定量地验证了动态流式处理、分阶段束搜索等策略的有效性。

第五章提出总结与展望，探讨了支持过滤、支持 GPU 动态更新和联合算法设计的研究方向。

第二章 背景介绍和相关工作

本章将从向量检索的概念出发，讨论当前主流的向量检索算法类型；接着，在介绍 GPU 的相关知识后，列举使用 GPU 加速的近邻图 ANNS 系统以及相关工作。

第一节 向量检索与近似最近邻搜索

在现代推荐系统^[3-4]与搜索引擎^[1-2]中，向量检索已成为实现深层语义匹配的核心技术。随着深度学习的普及，文本、图像、视频等非结构化的多模态数据可以通过各类嵌入模型（Embedding Model）被映射为高维稠密向量。这一过程将抽象的数据内容转化为向量空间中的坐标点，从而使得对多模态数据的相似性查找转化为向量空间中的最近邻搜索问题。

形式化来说，向量检索本质上是一个 K 最近邻（ K -Nearest Neighbor, KNN）问题。给定一个 n 维向量数据库 $\mathcal{P} \subset \mathbb{R}^n$ 和距离度量函数 $D(\cdot, \cdot)$ ，对于查询向量 $q \in \mathbb{R}^n$ ，目标是找到一个子集 $S \subset \mathcal{P}$ （其中 $|S| = k$ ），使得：

$$\forall p \in S, \forall p' \in \mathcal{P} \setminus S, \quad D(q, p) \leq D(q, p') \quad (2.1)$$

不同的上游嵌入模型在训练时采用了不同的损失函数，这决定了其生成的向量空间适用的相似性度量标准。在实际应用中，常见的距离度量方式如下：

- **欧氏距离 (Euclidean Distance / L_2):** 最直观的空间距离，数值越小表示越相似，广泛应用于计算机视觉（CV）领域的图像特征检索。计算公式为：

$$D_{L2}(a, b) = \|a - b\|_2 \quad (2.2)$$

- **内积距离 (Inner Product / IP):** 衡量向量在方向上的投影强度。在推荐系统中，用户和物品的交互概率常通过最大化内积来建模。其计算公式为：

$$D_{IP}(a, b) = \langle a, b \rangle = a^\top b \quad (2.3)$$

- **余弦相似度 (Cosine Similarity):** 衡量两个向量在方向上的夹角一致性，对绝对模长不敏感，常用于 NLP 领域的语义匹配。通常通过将向量归一化后

计算内积得到：

$$D_{Cos}(a, b) = \frac{a^T b}{\|a\| \|b\|} \quad (2.4)$$

由于精确的 KNN 搜索需要穷举计算查询向量与数据库中所有向量的距离，在大规模数据集下其计算开销是不可接受的。因此，实际的工业级搜索引擎广泛采用近似最近邻搜索（Approximate Nearest Neighbor Search, 简称 ANNS）^[7]。ANNS 旨在高维向量空间中快速查找近似的最近邻，其核心思想是通过构建特定的索引结构来剪枝搜索空间，从而以牺牲微小的检索精度为代价，换取查询效率的数量级提升。对 ANNS 系统的评估通常围绕检索精度、查询效率和资源消耗等方面展开。以下是若干核心评估指标：

召回率 (Recall@K) 是评估 ANNS 检索质量最关键的指标，它衡量了算法返回的近似结果与真实最近邻结果的重合程度。其定义为在返回的前 K 个结果中，包含真实最近邻（Ground Truth）的比例：

$$\text{Recall@K} = \frac{|N_{GT} \cap N_{ANNS}|}{K} \quad (2.5)$$

其中 N_{GT} 表示真实的 Top-K 最近邻集合， N_{ANNS} 表示算法返回的 Top-K 结果集合。

查询延迟 (Latency)：指处理单次查询所需的耗时，直接影响用户体验。除了平均延迟外，工程上常关注长尾延迟指标，如 P95 和 P99（分别表示 95% 和 99% 的查询在此时间内完成）。

查询吞吐量 (QPS)：指系统每秒能够处理的查询请求数量，反映了系统的并发处理能力。

距离计算次数：这是一个与硬件无关的效率指标，通常与查询延迟呈正相关，有些研究会将其用于衡量算法的计算复杂度。

内存占用：指索引结构驻留内存的大小。由于现代 ANNS 系统通常处理百万级（1M+）甚至十亿级（1B+）的数据量，内存效率是内存受限场景下的关键制约因素。

索引构建时间：指从原始数据生成 ANNS 索引所需的时间，受算法复杂度、硬件性能及构建参数（如连接度、候选集大小）的共同影响。

典型的 ANNS 系统通常由索引构建与查询服务两大功能模块构成，图 2.1 展示了 ANNS 系统的架构概览。

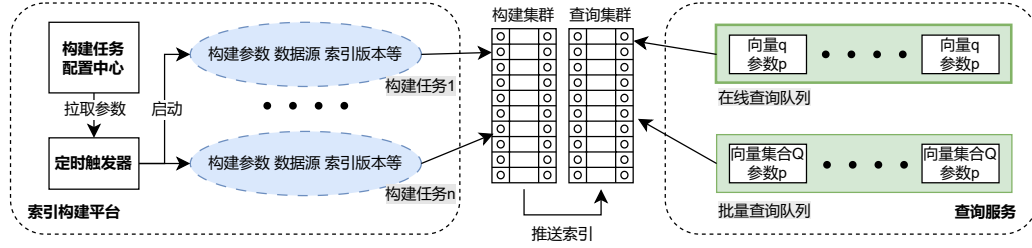


图 2.1 通用 ANNS 系统架构图

在索引构建阶段，系统需要根据具体应用场景的需求，在召回精度、检索性能及资源开销之间进行权衡，进而生成适配的定制化索引结构。查询服务模块则负责处理实际的检索请求，它通常兼容两种模式：既支持对延迟敏感的流式在线查询，也能处理对吞吐量要求较高的离线批量查询。本文的研究工作将重点聚焦于查询服务模块的关键技术优化。

第二节 ANNS 算法介绍

ANNS 算法的演进呈现出多元化的技术路径，目前常见的 ANNS 算法类型可以大体分为：基于近邻图的方案^[8-11]、基于量化的方案^[12-15]、基于树的方案^[17-18]和基于局部敏感哈希的方案^[7,16]。其中，近邻图算法在召回率与检索性能的综合表现上普遍优于其他类型；而量化方法则具有极高的灵活性，能与近邻图、倒排文件索引等结构有效结合，在压缩存储空间方面具有显著优势。在工业界的代表有 Facebook 的 Faiss 代码库^[19]，以及以 mlivus 为代表一系列支持向量检索数据库^[20-21]，它们实现了市面上主流的各类算法，方便开发人员集成到系统当中。在本节中会重点介绍近邻图和量化两大主流算法类型，涵盖多个常见的经典算法。

一、基于近邻图的方法

近邻图是本文重点研究的对象，其核心思想是将高维空间中的数据点连接成拓扑图结构，通过贪婪搜索策略来逼近最近邻。近年来的研究表明，基于近邻图的算法相较于其他类型的算法，在召回率与查询延迟的权衡上表现出显著优势。经典的近邻图结构根据选边策略的不同，主要可分为以下四类^[28]：

- **德劳内图 (Delaunay Graph, 简称 DG)**：确保任意两个相连节点 u, v 构成的边是某个圆的直径，且该圆内不包含图中其他任何节点。DG 保证了良好

的连通性和最近邻的包含性，但在高维空间中构建极其困难。

- **相对邻域图 (Relative Neighborhood Graph, 简称 RNG):** 对于任意边 (u, v) ，以 u 和 v 为圆心、 $\|u - v\|$ 为半径的两个超球体的交集（Lune 区域）内不能包含其他节点。
- **K-近邻图 (K-Nearest Neighbor Graph, 简称 KNNG/KNN):** 每个节点仅连接距离其最近的 K 个邻居。KNNG 构建简单，但容易产生孤立子图，缺乏全局连通性。
- **最小生成树 (Minimum Spanning Tree, 简称 MST):** 在保证全图连通的前提下，使得所有边的权重之和最小。

接下来重点介绍三种经典代表性算法：HNSW、NGT 与 NSG。

（一）HNSW

HNSW^[8-9] 是当前基于近邻图的最先进索引算法之一，其核心设计灵感源于跳表与小世界网络理论的结合。HNSW 通过在构图和搜索两个层面的精心设计，实现了高召回率与低延迟的平衡。

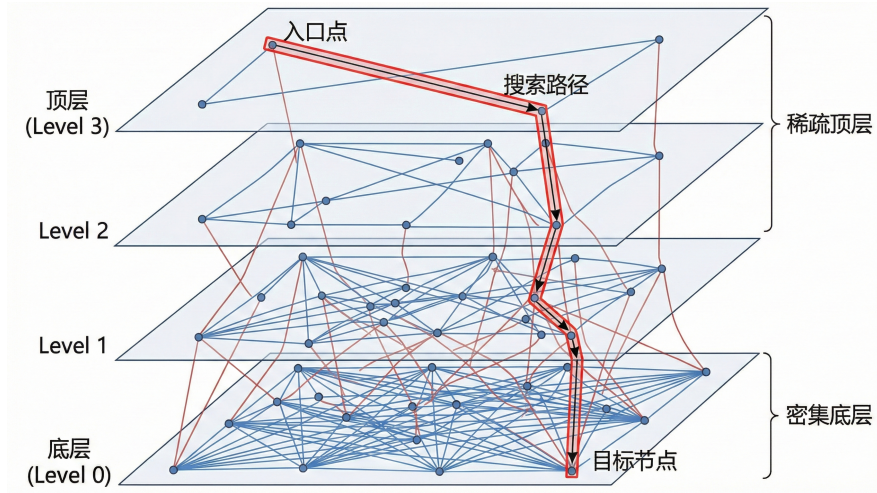


图 2.2 HNSW 结构示意图

从拓扑结构上看，HNSW 构建了一个多层级的金字塔式图结构（如图 2.2 所示），记为 $\{L_0, L_1, \dots, L_{top}\}$ 。其中底层 L_0 包含数据集中的所有节点，构成了高精度的基准图；随着层级 l 的升高，节点数量呈指数级递减，形成稀疏的“高速公路”层。对于每一个新插入的节点 q ，算法依据指数衰减概率分布为其随机分

配一个最高层级 l_{max} :

$$l_{max} = \lfloor -\ln(\text{uniform}(0,1)) \cdot m_L \rfloor \quad (2.6)$$

其中 $\text{uniform}(0,1)$ 为均匀分布随机变量, m_L 为控制层级密度的归一化因子 (通常设为 $1/\ln(M)$)。这种概率生成机制保证了图结构在宏观上具有类似于跳表的对数连通性。在构建策略上, 每层图中需新加入的点执行如算法 1 所示的查找过程, 获取其需要连接的候选邻居。然后通过启发式邻居选择策略淘汰部分候选邻居, 具体来说, 在连接节点 u 与候选邻居 v 时, 算法不仅考虑二者的绝对距离 $\|u - v\|_2$, 还强制检查 v 是否比 u 现有的邻居更近。这种策略本质上是对相对邻域图的近似, 通过优先保留指向不同方向的边, 有效抑制了“长边”的冗余, 增强了图的导航性。

Algorithm 1 单层图搜索算法

Require: 查询向量 q , 入口节点 ep , 目标数量 K , 搜索宽度 ef

Ensure: 查询向量 q 的 K 个最近邻

```

1: 初始化集合  $\mathcal{W}, \mathcal{C}, \mathcal{V} \leftarrow \{ep\}$ 
2: while  $\mathcal{C} \neq \emptyset$  do
3:    $c \leftarrow \mathcal{C}$  的距离最近未访问点
4:   if  $\text{Dist}(c, q) > \text{Dist}(\text{Max}(\mathcal{W}), q)$  then
5:     break 候选区最近的节点比结果区最远的还差, 停止搜索
6:   end if
7:   for all 未访问邻居  $e \in \mathcal{N}(c) \setminus \mathcal{V}$  do
8:      $\mathcal{V} \leftarrow \mathcal{V} \cup \{e\}$ 
9:     if  $\text{Dist}(e, q) < \text{Dist}(\text{Max}(\mathcal{W}), q)$  or  $|\mathcal{W}| < ef$  then
10:       $\mathcal{C} \leftarrow \mathcal{C} \cup \{e\}; \mathcal{W} \leftarrow \mathcal{W} \cup \{e\}$ 
11:      if  $|\mathcal{W}| > ef$  then
12:         $\text{RemoveMax}(\mathcal{W})$ 
13:      end if
14:    end if
15:  end for
16: end while
17: return  $\text{Top-K}(\mathcal{W})$ 

```

在检索过程中, HNSW 搜索始于最高层 L_{top} 的入口点, 算法在当前层贪婪地移动到距离查询向量 q 最近的邻居, 直至达到该层的局部最优点, 随即“降落”至下一层, 这一过程在稀疏的高层能够以极少的步数快速逼近目标区域。当搜索路由降落至最底层 L_0 时, 算法执行基于候选列表的图搜索, 旨在通过扩展

搜索范围来获取高质量的 Top-K 近邻。其核心逻辑如算法 1 所示。该过程维护两个关键集合：一个是动态结果集 $\mathcal{W}$ （用于存储当前找到的最优节点，容量受参数 ef 控制），另一个是候选集 $\mathcal{C}$ （用于存储待扩展的节点，通常实现为最小堆）。算法从入口点开始，不断从候选集 $\mathcal{C}$ 中取出距离查询向量 q 最近的节点 c 进行扩展。若 c 的距离已经远于结果集 $\mathcal{W}$ 中最差的节点，说明当前路径已无更优解，随即剪枝停止。否则，算法遍历 c 的邻居，将未访问且具有潜力的邻居加入 $\mathcal{C}$ 和 $\mathcal{W}$ 。通过调整超参数 ef ($ef \geq K$)，算法可以在搜索速度与召回率之间进行灵活权衡。

（二）NGT

NGT^[10] 是由雅虎开发的开源向量检索库。与 HNSW 的分层结构不同，NGT 致力于在单层图上通过优化边的连接来逼近德劳内图（DG）的特性。NGT 的构建过程分为两个阶段：首先构建一个近似 K-近邻图，通过树结构或哈希表快速初始化邻居关系；随后执行关键的路径调整与范围搜索优化。

在优化阶段，NGT 尝试为图中的节点添加捷径或移除冗余边。其核心剪枝逻辑近似于 RNG 图的定义：对于节点 u 的两个邻居 v 和 w ，如果满足 $d(u,v) > \max(d(u,w), d(w,v))$ ，则说明 w 位于 u 和 v 之间的路径上，边 (u,v) 可被视为冗余边而移除。

此外，NGT 引入了动态的种子节点搜索策略，在查询开始时通过从多个种子点出发来缓解单层图容易陷入局部最优的问题。凭借高度优化的图拓扑结构，NGT 在高维稠密向量数据集上展现出了极高的检索性能。

（三）NSG

NSG^[11] 是针对传统 K-近邻图搜索路径长、连通性差等问题提出的改进算法。其核心目标是构建一个稀疏但连通性极强的单调相对邻域图 (MRNG) 的近似图。

NSG 预先计算数据集的近似质心作为全局固定的导航点（入口点）。这确保了所有搜索都从图的中心位置开始，从而降低了平均搜索半径。在选择邻居时，NSG 考虑邻居分布的均匀性，采用极其严格的剪枝策略。对于节点 u 和候选邻居 v ，只有当 v 与 u 之间不存在另一个节点 p 使得 p 同时更接近 u 和 v 时，才保留边 (u,v) 。这种策略极大地减少了图的出度，从而缩小了索引体积并提升了内存访问效率。在构建初期，NSG 使用深度优先搜索来检测并连接那些在 KNN

中可能存在的孤立子图，从而显著增强了图的全局连通性。

实验表明，NSG 在保持高召回率的同时，生成的索引体积通常远小于 HNSW，非常适合内存受限的场景。

二、基于聚类与量化的方法

（一）乘积量化

乘积量化 (Product Quantization, 简称 PQ)^[12] 是一种基于向量子空间分解的高效压缩算法，旨在利用高维数据的局部结构特性来优化存储与计算。如图2.3所示，PQ 将 D 维原始向量空间分解为 M 个低维子空间。对于输入向量 x ，算法将其切分为 M 个子向量 $x = [x_1, \dots, x_M]$ ，并在每个子空间内独立使用 K-Means 聚类训练一个包含 K 个质心的码本 C_m 。原始向量最终被表示为 M 个子质心的索引元组，从而实现高压缩比。

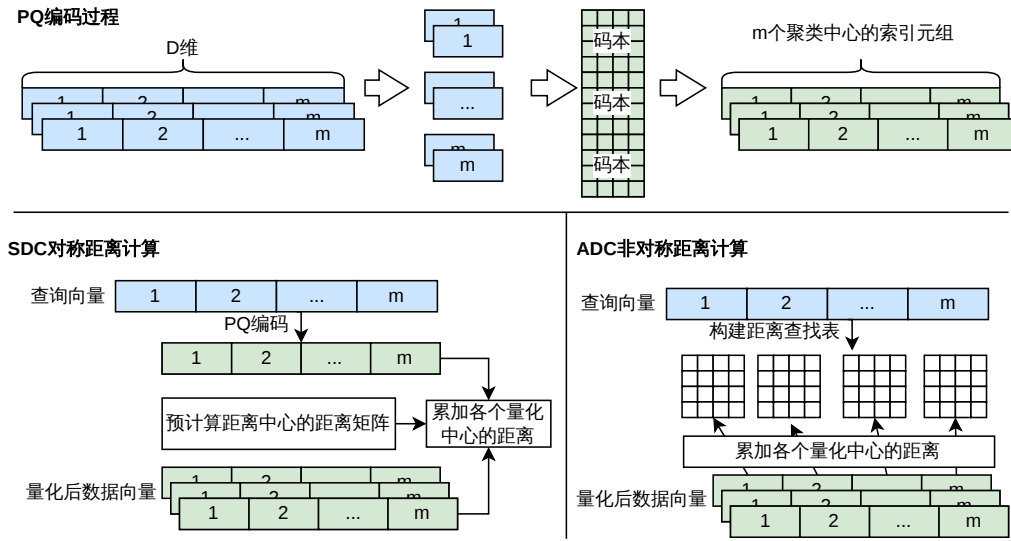


图 2.3 乘积量化构建与两种查找方式示意图

在距离计算模式上，PQ 支持对称距离计算 (Symmetric Distance Computation, 简称 SDC) 和非对称距离计算 (Asymmetric Distance Computation, 简称 ADC)。

在 SDC 模式的检索中，查询向量 q 和数据库向量 x 均被量化编码。距离计算在两个编码的聚类中心之间直接进行，因此可以直接预先穷举计算每个子空间中，所有聚类中心之间的距离，然后之间查表获取。这种方式虽然计算速度最快，但由于双方都存在量化误差，检索精度通常较低。

在 ADC 模式的检索中, PQ 通过预计算距离查找表 (Lookup Table, 简称 LUT) 显著加速距离度量。具体而言, 系统预先计算查询向量 q 的各子向量 q_m 与其对应码本 C_m 中所有质心的距离。此时, 查询向量 q 与数据库中编码向量 x 的近似欧氏距离无需进行浮点运算, 仅需通过查表并累加即可得出

$$\|q - x\|_2^2 \approx \sum_{m=1}^M \|q_m - c_{i_m}\|_2^2 \quad (2.7)$$

其中 c_{i_m} 为 x_m 在第 m 个子空间中对应的量化质心。该方法将昂贵的距离计算转化为高效的查表求和操作, 在保证一定精度的前提下, 极大提升了检索吞吐量。

(二) 标量量化

标量量化 (Scalar Quantization, 简称 SQ)^[29]旨在通过降低单维度数值精度来缓解大规模向量检索中的存储与带宽压力。SQ 独立地处理高维向量的每一个维度, 忽略维度间的统计相关性。如图所示2.4, 对于 D 维向量 x , 算法统计训练集在每一维度上的数值分布边界 $[l_j, u_j]$, 并将该范围均匀划分为 2^B 个离散区间 (通常 $B = 8$)。原始的高精度浮点数值 $x^{(j)}$ 依据其所在的区间被映射为唯一的整数索引, 从而将单维度的存储成本从 32-bit 显著压缩至 8-bit 或更低。此外, 优化的 SQ 算法针对数据中常见的离群值问题采用了统计截断策略, 即根据数据分布的百分位确定量化边界, 使有限的量化桶能更集中覆盖数据密集区域。

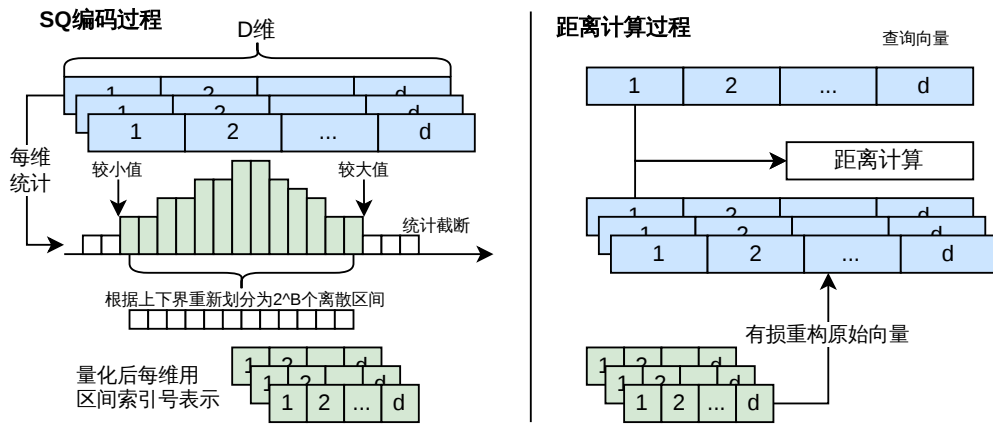


图 2.4 标量量化构建与查找方式示意图

在检索阶段, 为了最大限度地减少精度损失, SQ 通常也采用非对称距离计算策略。该策略保持查询向量 q 为原始浮点精度, 仅对数据库中的压缩向量进

行实时重构与距离估算：

$$\|q - x\|_2^2 \approx \sum_{j=1}^D \left(q^{(j)} - \hat{x}^{(j)} \right)^2 \quad (2.8)$$

其中 $\hat{x}^{(j)}$ 为基于存储的整数索引恢复出的近似分量。由于 SQ 的编解码过程仅涉及简单的线性变换，且数据内存布局规则，极易利用现代 CPU/GPU 的 SIMD（单指令多数据）指令集进行并行加速。

（三）RaBitQ 量化

为了进一步挖掘量化算法在现代硬件上的性能潜力并解决传统方法缺乏理论保障的问题，Gao 等人提出了 RaBitQ^[13-14]。这是一种结合了随机投影理论与位级量化的高效算法，旨在为高维向量的近似最近邻搜索提供具有严格理论误差上界的距离估计。

不同于 PQ 通过启发式聚类来划分自空间，RaBitQ 采用了一种基于几何的随机化构造策略。其核心流程包含两步：首先，利用一个随机正交矩阵 P 对归一化后的数据向量进行旋转；随后，对变换后的向量进行二值化处理（取符号位），将其映射为 D 维的比特串。这种设计构造了一个由超立方体顶点经随机旋转后组成的码本，使得量化后的向量在单位超球面上均匀分布。相较于传统方法的复杂码本训练，RaBitQ 的索引构建仅需存储随机矩阵种子，极大地降低了空间与时间成本。

RaBitQ 的最大贡献在于解决了传统量化方法距离估计有偏且无误差上界的缺陷。这意味着随着向量维度的增加，RaBitQ 的估计精度会自然提升，从而避免了传统量化在某些数据分布下失效的情况。此外，该误差上界还被直接用于指导重排序阶段的候选集截断，无需像 PQ 那样依赖经验调参。

在计算层面，RaBitQ 支持将浮点内积转化为高效的位逻辑运算，其计算速度比基于查表的 PQ 实现快约 3 倍，极大地降低了单次查询的延迟。对于批量查询场景，RaBitQ 同样支持利用 AVX-512 等 SIMD 指令集进行向量化加速，在保持高精度的同时达到了与现有最快量化实现相当的吞吐量。

（四）倒排文件索引

倒排文件索引 (Inverted File Index, 简称 IVF) 的核心思想是利用“分而治之”的空间划分策略，将高维空间中的全局搜索转化为局部子空间内的搜索，从而显著降低计算复杂度。

在索引构建阶段，IVF 引入粗糙量化器将向量空间划分为 K 个单元，每个单元由一个质心 c_i 及其对应的倒排列表 $\mathcal{L}_i$ 表示。数据集中的向量 x 仅被存储在与其最近质心对应的列表中。在检索阶段，给定查询 q ，算法不再遍历全量数据，而是执行两步策略：首先在粗糙量化器中找到距离 q 最近的 w 个质心（ w 即超参数 `nprobe`）；随后，仅在这些被选中的倒排列表集合 $\mathcal{S}_q$ 中进行精细搜索：

$$\tilde{x}^* = \arg \min_{x \in \mathcal{S}_q} \|q - x\|_2 \quad (2.9)$$

这种方法有效地剪枝了绝大部分无关的搜索空间，在召回率与检索速度之间取得了良好的平衡。

在大规模场景下，为了进一步压缩内存占用并加速距离计算，IVF 通常与量化技术结合使用。例如 IVFADC^[12] 结合残差量化^[30]，倒排列表中存储的不再是原始向量，而是向量与其所属质心的残差。该残差向量通常通过 PQ 量化进行编码存储。在检索时，系统利用查表法快速估算“查询向量与残差”的距离。工业界的代表性系统 Faiss 正是基于 GPU 高效实现了 IVF 与 PQ 的结合，通过对残差的量化处理，成功支撑了十亿级数据的实时检索。

第三节 GPU 架构与 CUDA 计算模型

英伟达图形处理器（Graphics Processing Unit, 简称 GPU）^[22] 凭借其海量的计算核心与高吞吐量的内存子系统，已从早期的专用图形渲染设备演变为现代高性能计算与深度学习领域的主流加速平台。在 ANNS 领域，目前有越来越多的工作与 GPU 结合，在性能上相比于 CPU 有着数量级的提升。为了方便后续理解，在本节我们简要介绍 GPU 的相关背景知识。

与 CPU 旨在优化单线程延迟和复杂控制流的架构设计不同，GPU 采用了以数据吞吐量为核心的众核设计理念，致力于通过大规模线程级并行来隐藏内存访问延迟。图 2.5 展示了 NVIDIA 通用计算 GPU 的典型架构。

一、硬件架构与执行模型

在 NVIDIA GPU 的现代硬件架构（如 Ampere、Hopper）中，多个流式多处理器（Streaming Multiprocessor, 简称 SM）构成了计算的基本模块。每个 SM 内部高度集成了用于处理标准算术运算的 CUDA 核心（FP32/INT32 Cores），以及自 Volta 架构引入的、专用于加速矩阵乘加运算的张量核心（Tensor Cores）。这

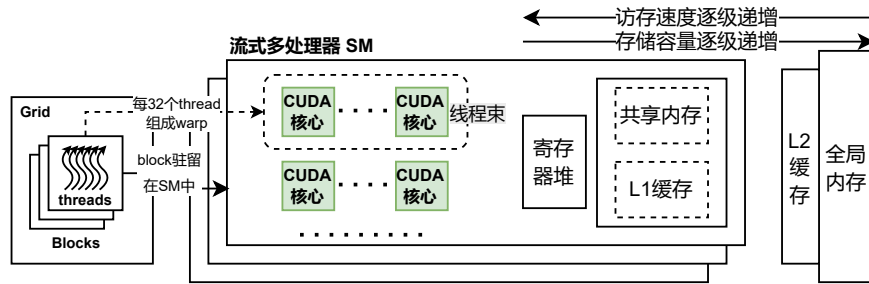


图 2.5 通用计算 GPU 架构示意图

种异构核心设计使得 GPU 能够兼顾通用科学计算与特定领域的混合精度计算需求。

在指令执行层面，GPU 采用 Single Instruction, Multiple Threads, SIMT) 架构。SM 内的 Warp 调度器 (Warp Scheduler) 将 32 个并行线程编组为一个线程束 (Warp)。Warp 是 GPU 硬件调度的最小单位，束内的所有线程在同一时钟周期内执行相同的指令。这种机制极大地简化了控制逻辑的芯片面积占用，但同时也引入了线程束分化的挑战：当束内线程因条件分支 (如 if-else) 进入不同执行路径时，硬件必须串行化执行各个分支，从而导致并行效率降低。此外，GPU 依靠极其快速的零开销上下文切换机制，在当前 Warp 等待内存数据时，迅速切换至就绪的 Warp 继续计算，从而有效掩盖了长延迟的显存访问操作。

二、层次化存储子系统

为了弥合计算单元极高的算力与受限的显存带宽之间的“内存墙”问题，GPU 设计了层次分明的存储架构：

- **全局内存 (Global Memory):** 位于最底层，通常由高带宽显存 (如 GDDR6X 或 HBM) 组成。尽管其容量最大，但访问延迟高达数百个时钟周期。为了最大化带宽利用率，GPU 内存控制器要求线程束内的内存访问满足合并访问规则，即相邻线程访问连续对齐的内存地址，以便将多次离散请求合并为单一的内存事务。在全局内存上，所有 SM 还共享统一的 L2 缓存，作为连接计算核心与显存的一道高速桥梁。
- **共享内存 (Shared Memory) / L1 缓存:** 为了提升数据复用率，每个 SM 内部配备了用户可编程的共享内存和硬件管理的 L1 缓存。共享内存作为一种低延迟的暂存空间，允许同一线程块内的线程进行高效的数据交换与协作，通过避免重复访问全局内存，显著提升了系统吞吐量。

- **寄存器堆**：这是访问速度最快的存储单元，但资源极其有限。每个线程消耗的寄存器数量直接决定了 SM 上可同时驻留的活跃 Warp 数量，进而直接影响 GPU 隐藏延迟的能力。

三、CUDA 编程模型与软件抽象

在软件层面，NVIDIA 的 CUDA^[31]编程语言提供了成熟的通用并行计算范式。CUDA 基于异构计算模型，将系统抽象为负责逻辑控制与串行任务的主机（Host/CPU）与负责高密度并行计算的设备（Device/GPU）。

CUDA 将计算任务通过内核函数（Kernel）的形式卸载至 GPU，并定义了三级逻辑并行层次结构：

1. **网格（Grid）**：代表一次内核启动的全部任务空间，被映射到整个 GPU 设备上。
2. **线程块（Block）**：是网格的子划分，每个线程块被调度到一个物理 SM 上独立且驻留执行。
3. **线程（Thread）**：是执行的最小单元。同一线程块内的线程可以通过共享内存进行低延迟通信，并利用同步原语（如 `__syncthreads()`）保证数据一致性。

这种抽象不仅实现了软硬件解耦，使得程序能够透明地扩展至不同规模的 GPU，还支持更高级的并发机制。例如，通过 CUDA 流（CUDA Streams），开发者可以实现多个 GPU 核函数的并发执行，以及计算任务与主机到设备（H2D/D2H）数据传输的流水线重叠，从而进一步榨取硬件性能，最大化系统吞吐量。

第四节 GPU 加速 ANNS 的研究现状

一、GPU 加速近邻图索引

（一）SONG

SONG^[23]是学术界首个在 GPU 上实现通用图索引近邻搜索的完整系统框架，其核心贡献在于解决了传统图算法与 GPU 硬件特性之间的不匹配问题。在 CPU 时代，以 HNSW 为代表的图算法依赖于细粒度的贪婪搜索，即“查找一个点—计算距离—更新队列—再查找下一个点”，这种强串行依赖性导致 GPU 的数千个计算核心无法发挥并行优势，且频繁的非规则内存访问严重拖累了性能。

为了打破这一瓶颈，SONG 提出了一种适用于 GPU 的流水线搜索范式。在

每轮搜索迭代中，首先进行候选点定位，从当前的候选队列中一次性提取多个待扩展节点，而非单个节点；其次是批量距离计算，利用 GPU 的 Warp 级并行归约技术，同时计算查询向量与所有待扩展邻居的距离，极大地提升了计算密度；最后统一排序候选队列，维护好下一轮拓展的需要。

此外，针对 GPU 片上共享内存容量极小（通常仅几十 KB）的限制，SONG 创新性地引入了布隆过滤器 (Bloom Filter) 和布谷鸟过滤器 (Cuckoo Filter) 来替代传统的哈希表进行已访问节点的去重。这些结构以极低的空间占用实现了高效的成员查询，有效减少了对慢速全局显存的访问。

实验表明，SONG 在百万级数据集上利用大型批处理将搜索速度有数量级的提升，确立了 GPU 图搜索的基本范式，为后续相关工作提供了参考和借鉴。可惜的是，SONG 的并行架构专注于理论吞吐量的提升，缺乏对低延迟和实际使用的进一步讨论。

（二）GANNS

GANNS^[24]深入剖析了 SONG 等早期工作在性能上的瓶颈。他们发现，虽然向量距离计算已被高度并行化，但维护有序队列和哈希表等非计算密集型操作仍占据了 50%-90% 的总耗时。这些操作涉及大量的条件分支和随机访存，是 GPU 最不擅长的任务。为此，GANNS 提出了一套懒惰交互策略。

在索引构建方面，GANNS 设计了一种基于分治的快速 HNSW / NSW 图构建算法。使用 GPU 并行构建 KNN 子图，然后递归地将子图逐个合并。在合并单个子图过程中，GANNS 分两阶段添加正向边和反向边，利用全局排序和前缀和，高效处理了图合并过程中反向边的冲突与更新问题，使得在 GPU 上构建高质量图索引的速度比传统 CPU 单线程方案快 40 至 50 倍。不过由于并行构建的实现难度，GANNS 构建的 NSW 图索引并不支持原生 HNSW 的三角裁边策略。

在搜索过程中，GANNS 引入了懒惰更新和懒惰检查机制。与传统算法每次迭代都严格维护队列有序性不同，GANNS 使用固定长度的无序数组暂存新生成的候选点，并利用 GPU 高效的双调排序网络进行批量排序和归并。这种设计规避了细粒度的串行插入操作，最大化了指令流水线的吞吐量。同时，GANNS 移除了用于实时去重的全局哈希表，允许在中间步骤进行少量的冗余距离计算，仅在最终结果合并阶段通过并行二分查找进行延迟去重。这种“以计算换规则化访存”的策略，显著提升了显存带宽的有效利用率。图2.6形象化地展示了 GANNS 的搜索模式。和 SONG 类似，GANNS 也仅优化峰值吞吐量，端到端查

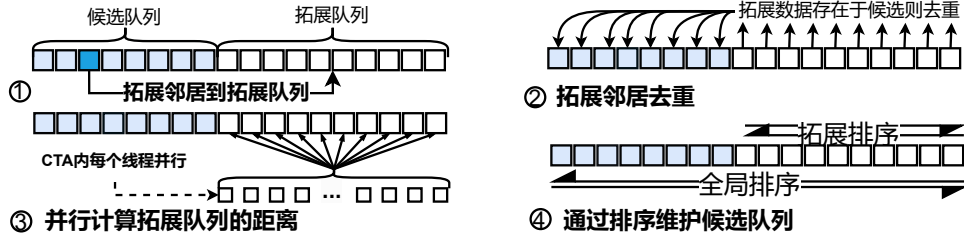


图 2.6 GANNS 搜索方法：每轮迭代中，GANNS 挑选出当前最近未访问点（1）；然后将对应邻居放置到扩展队列，过滤掉已经存在于候选队列的点（2）；并行计算扩展队列中点的距离（3）；最后进行双调排序和双调合并，更新候选队列（4）。

询延迟较高，没有针对小批次的高并行架构。

（三）GGNN

针对大规模图索引构建耗时过长的问题，GGNN^[25] 提出了一套自底向上的分层构建策略（如图2.7所示），将图构建速度提升到了新的高度。

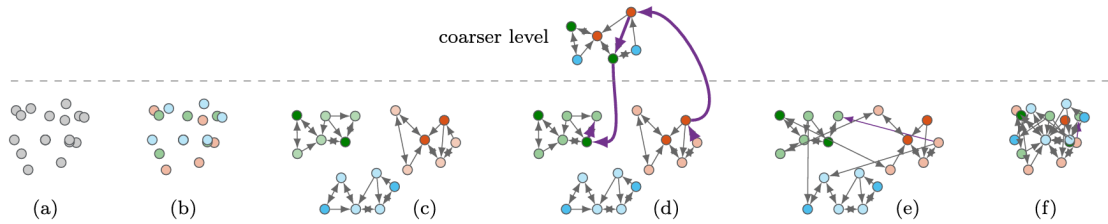


图 2.7 GGNN 构图过程：(a) 经过随机划分 (b) 后，为每个划分构建一个 KNN 图 (c)。选择部分节点构建一个较粗略的 kNN 图 (d)，用于在分片之间建立连接，这些连接用于将多个分片合并 (e)。最后添加反向边合并成最终的 kNN 图 (f)。

在搜索执行层面，GGNN 针对 GPU 的 SIMT 架构设计了适用于 CUDA 线程块的协同查询模型。具体来说，GGNN 使用了一种束搜索的模式，支持 GPU 并行在候选队列中同时拓展一批节点，然后并行地计算所有被拓展的邻居，最后基于双调排序来维护候选队列，这与 GANNS 的搜索过程2.6类似，GGNN 在此之上支持同时拓展多个邻居，它们的都使用了的拓展列表去重来代替的历史节点访问记录。GGNN 还提出了一种动态松弛因子的自适应停止准则，根据当前搜索到的第 k 个最近邻距离动态调整搜索半径，这种机制在保证 99% 以上高召回率的同时，能够比固定阈值策略更早地终止无效搜索，显著降低了长尾查询的延迟。

但是，GGNN 和 SONG 一样沿着高吞吐的目标优化，在实时低延时搜索上

并不占优势，无法在超越一个 CUDA 线程块的并行上限进行搜索。此外，并行拓展一批节点的模式虽然可以很好的适用于 GPU 架构，但是存在计算有效性问题，图搜索过程中大部分节点最终并不会进入到结果集，一定程度上放大了计算量。

（四）BANG

针对单卡 GPU 显存容量无法容纳十亿级图索引的瓶颈，BANG^[26] 提出了一种高效的 CPU-GPU 异构协同架构。类似于 DiskANN 的思路，BANG 采用“图驻留内存，计算卸载显存”的策略：它将庞大的 Vamana 图结构维护在 CPU 主存中，仅将通过乘积量化压缩后的向量数据常驻 GPU。

在检索过程中，BANG 利用 GPU 的高并行度进行基于压缩向量的距离估算，并通过 PCIe 总线按需从 CPU 获取邻居节点信息。为了缓解 PCIe 带宽瓶颈，BANG 设计了精细的流水线机制，利用 CUDA Streams 实现数据传输与内核计算的重叠，并引入 GPU 端的布隆过滤器来高效管理访问历史，从而在单卡 A100 上实现了相比同类方法 40 倍至 200 倍的吞吐量提升。

然而，BANG 依然沿用了传统的贪婪搜索策略，且依赖于有损的 PQ 量化进行粗排，在高召回率要求下可能面临精度损失。

（五）CAGRA

CAGRA^[27] 是目前 NVIDIA 推出的最先进的 GPU 图索引方案，其核心理念是“为硬件设计算法”。图2.8展示了 cagra 图索引的具体构建过程。

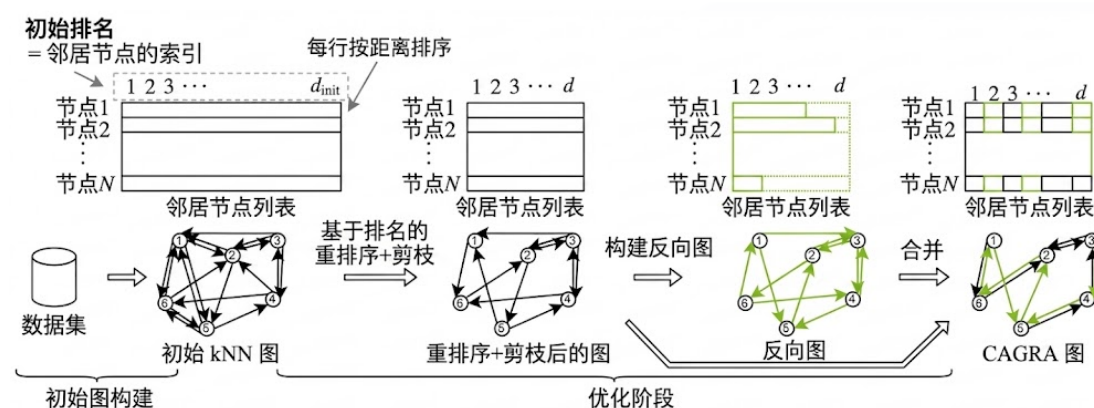


图 2.8 cagra 图索引构建过程：利用 NN-Descent 算法构建 kNN 初始图；然后每个邻居按照边长度排序，计算每条边可绕行数量后重排，剪枝掉排名靠后的边；获取反向边，合并到整张图，并将出度截断到统一长度。

CAGRA 的图索引不同于以往将 CPU 友好的图结构（如 HNSW）强行移植到 GPU，CAGRA 从底层重新设计了图的拓扑结构。它强制采用固定出度的单层有向图，这种高度规则化的结构可以直接通过偏移计算获得邻居列表，消除了邻接表访问时的负载不均衡和线程束分化。在图的优化阶段，传统图索引一般会使用一些剪枝策略。CAGRA 引入了基于秩的重排序技术来快速剪枝，该方法利用初始图中邻居列表的排序位置作为重要性的近似指标，替代了昂贵的精确距离重计算。此外，CAGRA 通过添加反向边增强了强连通分量，保证了极佳的遍历性能。

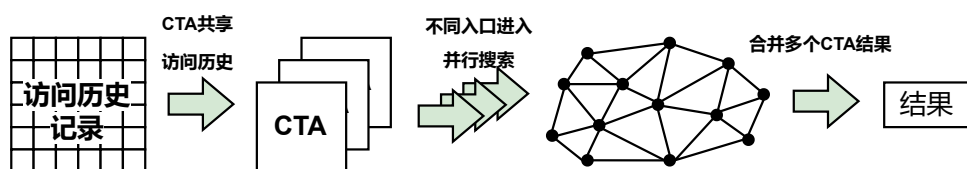


图 2.9 Multi-CTA 模式中 CTA 的协作过程，每个 CTA 都从不同的随机入口进入图索引，它们共享相同的访问历史，可以快速并行地搜索区域，最后各 CTA 将结果进行聚合。

在搜索算法上，CAGRA 针对不同的应用场景灵活设计了两种策略，基于下发的查询批次大小调整每个查询分配的计算资源数，可以应对高吞吐和低延迟的场景：

- **Single-CTA 模式：**面向高吞吐量的海量查询场景（使用大规模批处理）。每个查询由一个独立的线程块处理，利用片上共享内存中的可遗忘哈希表进行极速去重。该哈希表允许在填满后自动重置，以极小的空间代价换取了极高的缓存命中率。
- **Multi-CTA 模式：**面向低延迟的小批量或单查询场景（使用小型批处理）。由于 GPU 的单个 SM 内并行的线程数目有上限，因此更多的线程并行需要跨越 CTA，在 Multi-CTA 的设计中，一个查询被分发给多个线程块协同处理，通过软件级线程束拆分技术进一步提升并行度，不同 CTA 的协助搜索过程如图 2.9 所示。

论文实验数据表明，CAGRA 在保持 SOTA 级别召回率（95%+）的同时，其单查询延迟比 CPU 上的 HNSW 快 3.4-53 倍，在大批量吞吐量上则领先 33-77 倍，代表了当前 GPU 图搜索技术的最高水平。

二、GPU 加速非图索引

Johnson 等人提出的 Faiss^[19]是目前工业界最具影响力的向量检索引擎之一。针对十亿级的高维向量检索任务，Faiss 将倒排索引（IVF）与 PQ 量化相结合，并针对 GPU 架构进行了深度优化。其核心贡献在于解决了 GPU 在处理 Top- k 选择时的性能瓶颈：通过设计基于寄存器的 WarpSelect 和基于共享内存的 BlockSelect 原语，Faiss 极大地提升了在海量距离候选中筛选最近邻的效率。此外，Faiss 充分利用了 GPU 的多层存储器结构，实现了对量化码本和残差向量的高效并行访问，确立了 GPU 在大规模向量检索中的统治地位。目前的 Faiss 已演进为一个集成了多种经典算法（如 HNSW、IVF 等）的通用向量检索库。

为了进一步解决十亿级数据在 GPU 有限显存中的存储与访问难题，Chen 等人提出了 Robustiq^[32]。该工作指出，现有的量化方法（如 IVFADC）在 GPU 上面临着严重的非合并内存访问问题，导致带宽利用率低下。Robustiq 提出了一种细粒度的数据布局策略，将粗粒度的倒排列表量化与细粒度的残差量化相结合。在保证检索精度的同时，该策略通过平衡量化误差与内存访问开销，显著优化了 GPU 的显存带宽利用率。实验表明，在处理超出单卡显存容量的大规模数据集时，Robustiq 展现出了比 Faiss 更优的吞吐量与延迟平衡。

针对通用 GPU 在执行乘积量化检索时存在的访存效率低下与能效瓶颈，Lee 等人提出了专用硬件加速器架构 Anna^[33]。该架构摒弃了通用的指令执行模式，设计了专用的数据流流水线，通过定制的簇处理模块和相似度计算模块硬化执行 Faiss 等算法的核心逻辑。此外，Anna 引入了一种内存流量优化机制，利用批处理查询中同一聚类数据的复用性，显著降低了对片外内存的访问需求。这种软硬协同的设计最终实现了相比传统 CPU 和 GPU 方案在能效与吞吐量上的数量级提升。

三、其他相关研究

针对内存容量无法容纳的十亿级超大规模数据集，研究基于磁盘的 ANNS 具有重要意义。DiskANN^[34]是该领域的代表性工作，它设计了 Vamana 图结构，通过保留长边来显著减少搜索跳数，从而降低了随机磁盘 I/O 的次数，成功实现了在商用 SSD 上的高吞吐检索。

SPANN^[35]则提出了一种内存-磁盘混合策略，将聚类质心驻留在内存中以快速定位，而将详细的倒排列表存储在磁盘上，有效平衡了低延迟检索与大规

模存储成本之间的矛盾。

DF-GAS^[36] 针对图算法的不规则访存特性，在 FPGA 上设计了一种数据流架构，通过流水线化图遍历过程显著隐藏了内存延迟。

针对数据移动带来的开销，SmartANNs^[37] 利用 SmartSSD 技术实现了近存储计算，将部分检索任务卸载至 SSD 控制器，从而大幅减轻了主机 CPU 的负担。

CXL-ANN^[38] 利用新兴的 CXL (Compute Express Link) 互联技术，探索了在内存解耦架构下的大规模向量检索，为打破单机内存限制提供了全新的硬件思路。

FusionANNs^[39] 通过 CPU、GPU 和 SSD 的协同工作来优化内存使用效率。在十亿级数据规模下，具有高吞吐量、低延迟、低成本和高精度的优势。

第五节 本章小结

本章系统地梳理了向量检索的研究背景、核心算法及硬件加速技术。

首先，本章从向量检索的基本定义出发，介绍了主流的距离度量方式与系统评估指标，并详细探讨了当前两大类主流 ANNS 算法：基于近邻图的方法以其优异的召回率与性能权衡成为研究热点，而基于量化的方法则在压缩存储与计算加速方面展现出独特优势。

介绍完 GPU 相关架构后，本章重点综述了 GPU 加速 ANNS 的研究现状。从早期的 SONG 框架探索流水线并行，到 GANNS 和 GGNN 针对构建与搜索的深度优化，再到当前 SOTA 级别的 CAGRA 通过结合 GPU 架构的设计实现性能飞跃，以及 BANG 等工作对 CPU-GPU 异构架构的探索。本章的相关工作综述为本文后续提出的优化方案提供了理论基础与技术参照。

第三章 GPU 向量检索架构设计

第一节 研究动机

尽管当前基于近邻图的 GPU 向量检索算法在峰值吞吐量指标上已到达较高水平，但其在查询延迟方面的表现却普遍不尽如人意，难以满足搜索引擎等对实时性要求极高的在线服务场景。早期的代表性工作（如 SONG 和 GANNS）主要采取激进的大规模批处理策略，即在测试峰值性能时将海量查询聚合为一个超大批次提交给单个核函数执行。虽然这种方式能够充分饱和 GPU 的计算核心利用率，并通过多查询间的访存与计算重叠显著提升吞吐量。但是这在实际运用中需要等待大量的请求以供调度，无法满足在线搜索等业务对实时性的极高要求，会导致不可接受的端到端排队延迟。

近邻图查询算法的核心通常基于贪婪搜索，其过程具有强烈的顺序依赖性，这与 GPU 擅长的规则化、大规模数据并行计算模式冲突。值得注意的是，随着 GPU 算力的提升，CPU 中占据主导的高维向量距离计算开销被极大压缩。这一变化也让 GPU 近邻图查询中不规则的非计算密集型开销占比上升，成为新的瓶颈。为了缓解 GPU 在低延迟场景下的性能劣势，CAGRA 提出了一种以单个查询独占整个 GPU 核心资源的加速方案。其核心思想是将单个查询任务并行分发给多个 CTA，能够实现跨 SM 的并行，即 Multi-CTA 模式。在此模式下，各 CTA 独立执行近邻图遍历，通过共享全局已访问表来避免重复计算，并最终聚合各组结果。Multi-CTA 模式可推广至小批量（如 16、32、64 等批次）处理场景，旨在兼顾高吞吐量和查询延迟的需要。不过这种策略也引入了额外的局部分片结果合并开销，跨 SM 的结果聚合依旧需要全局显存的参与，抵消了并行计算带来的部分收益。并且 CAGRA 的 Multi-CTA 主要侧重于从算法角度探讨小批量处理的理论性能峰值，调度模式依旧是传统的阻塞式调度模型，而缺乏针对实际系统架构层面的优化。

接下来，本文分别从请求调度层面与算法执行层面来分析当前的 GPU 向量检索系统面临的挑战。

一、请求调度层面的批处理气泡

在传统的 GPU 批处理执行模型中，当一个包含多个查询的小批次被提交至 GPU 时，系统往往采用同步执行逻辑，必须等待整个批次内所有查询完成后才能进行下一步操作。本文通过实验观察发现，由于近邻图拓扑结构的非规则性，不同查询在图上的搜索路径长度（即跳数）存在显著差异，导致批次内存在严重的长尾延迟问题。

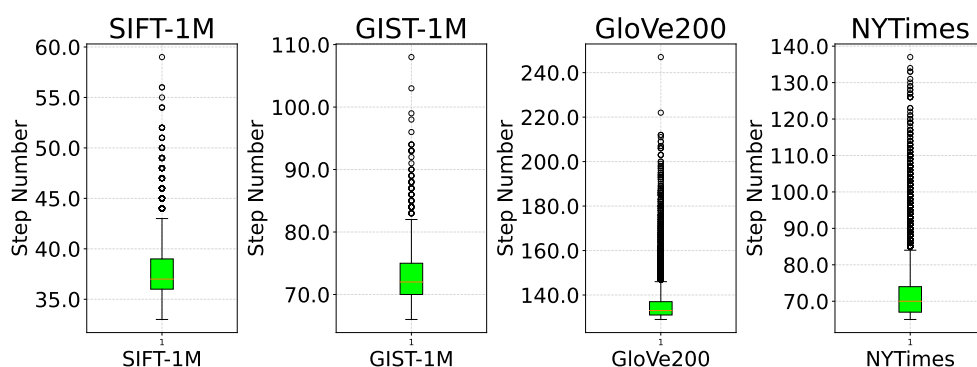


图 3.1 不同数据集下的跳数总体分布

为了量化这一现象，本文统计了不同数据集下查询过程中的跳数分布，如图 3.1 所示。实验数据表明，所有数据集的查询跳数均呈现显著的离散性，在极端情况下，最慢查询的跳数可达平均跳数的 147.9% 至 190.2%。即使在小批量情况下，查询之间的步数也存在明显的不均衡。图 3.2 显示了批次内的步数分布（批量大小为 32）。在不失一般性的情况下，我们为每个数据集选择了 8 个批次来展示结果。在 GIST-1M 数据集中，查询步数的分布范围大约在 70 到 100 之间。最慢的查询所需的步数比最快的查询多达 32.4%。

在这种情况下，整个批次的完成时间被迫取决于那个“最慢的查询”，从而拉高了整个批次的平均延迟。我们将这种因查询快慢不均导致的延迟拖累现象定义为“批处理气泡”。进一步的分析可以发现，批处理气泡问题在小批量处理场景下尤为严峻，主要体现在两个维度：

- **气泡频率增加：**在固定总查询量的前提下，采用小批量处理意味着需要提交更多的批次。由于每个批次都不可避免地存在长尾查询，这导致气泡出现的频率大幅增加，从而累积成为制约系统整体性能的关键瓶颈。
- **GPU 算力浪费：**在大批量场景下，过饱和的并发任务可以填补空闲时隙；但在小批量场景下，GPU 利用率本就不饱和，一旦批次内部分查询提前完

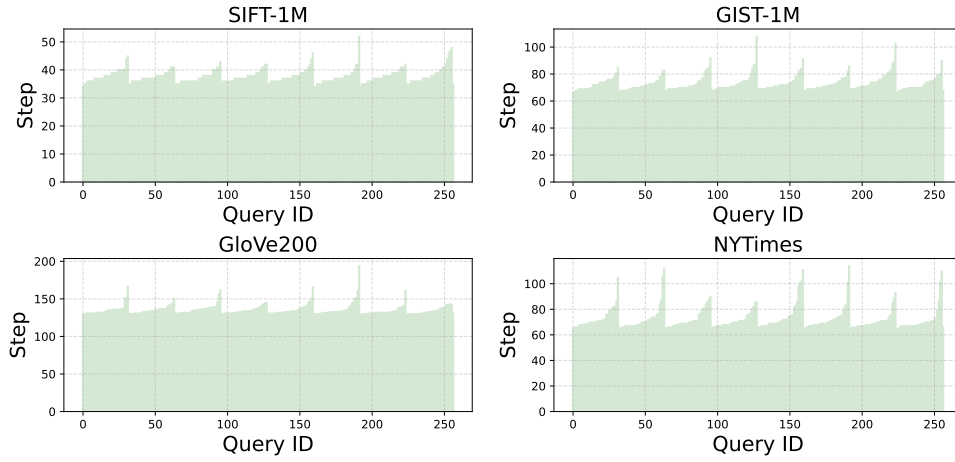


图 3.2 不同数据集下批次为 32 的跳数分布

成，其对应的计算核心将直接进入空闲状态，导致算力的显性浪费。

二、算法执行层面的性能瓶颈

除了调度层面的气泡问题，现有的 Multi-CTA 检索架构在具体的并行执行逻辑上也存在优化空间。我们可以将 Multi-CTA 划分为 CTA 内并行与 CTA 间并行两个层次，但在实际运行中，这两个层次均面临着显著的性能瓶颈：

在 CTA 内部，虽然向量距离计算利用 SIMT 特性和 Warp Shuffle 规约实现了高度并行化，但候选队列的维护操作却成为了新的瓶颈。每当有新邻居节点加入时，都需要与现有候选点进行排序和截断。如图 3.3 所示，在不同数据集的测试中，排序操作占据了总搜索开销的 19.9% 至 33.9%。即便采用了如 GANNS 中高效的并行数据结构维护策略，排序带来的计算延迟依然显著，这种非计算密集型的逻辑操作严重制约了单次查询的响应速度。

Multi-CTA 算法要求在搜索结束后，将分散在各 CTA 中的局部 TopK 结果合并为全局最终结果。这一过程面临双重挑战：

- **访存效率低：**各 CTA 有可能分布在不同的流式多处理器上，跨 CTA 的数据合并无法利用低延迟的片上共享内存，必须频繁访问高延迟的全局显存来交换数据，导致访存流水线阻塞。
- **并行度受限：**合并多个有序 TopK 列表的算法本身难以完全并行化。即使采用分治策略进行归并，随着归并层级的上升，活跃的计算任务逐渐减少，导致近半数的计算核心处于空闲状态。这种算力资源的浪费大幅降低了硬

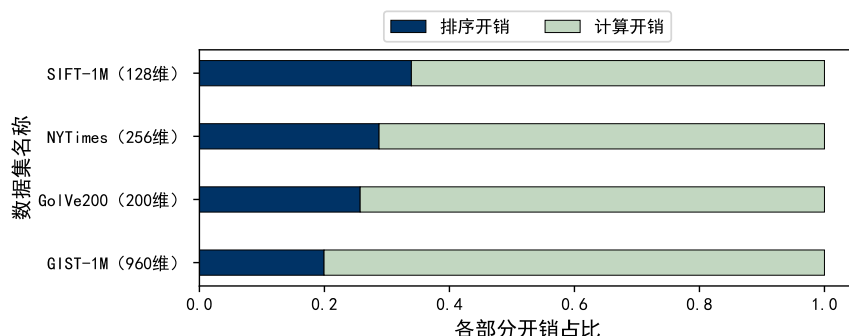


图 3.3 排序和距离计算开销占比

件利用率，抵消了多 CTA 并行带来的部分收益。

综上，如何消除调度层面的批处理气泡，优化 CTA 内的排序逻辑，并解决 CTA 间的高昂合并开销，是构建高效 GPU 近邻图 ANNS 亟待解决的核心挑战。

第二节 系统总体架构

针对前文分析中指出的调度层气泡与执行层效率低下的双重瓶颈，本文提出了一种面向低延迟场景的高性能 GPU 向量检索系统架构。

图 3.4 展示了该系统的整体架构概览，图中黑色序号标注了查询请求在系统各模块间的流转过程。关于查询执行的具体细节，将在后续章节进行详细阐述。该架构的核心设计包含两个方面：

针对调度层的查询气泡问题，我们提出了一种基于 **GPU 动态流式处理** 的调度架构。该策略摒弃了传统的同步批处理模式，将计算资源划分为多个逻辑独立的**槽位 (Slots)**。配合 GPU 上的持久化核函数技术，每个槽位能够独立接收任务并立即执行，无需等待批次内的其他查询同步。这种细粒度的调度机制彻底打破了批次边界，有效消除了因长尾查询导致的 GPU 核心空闲，显著减少了调度气泡。

针对并行检索算法的效率瓶颈，本文设计了一套 CPU-GPU 协同的搜索算法。该算法一方面通过分阶段策略降低了 GPU 在搜索过程中的排序维护频率，另一方面将逻辑复杂但计算量较小的 TopK 合并操作卸载至 CPU 端执行。这种设计确立了“GPU 专注并行计算，CPU 处理逻辑归并”的异构流水线，最大化了各硬件单元的效能。

在接下来两节中，我们将详细介绍相关设计。

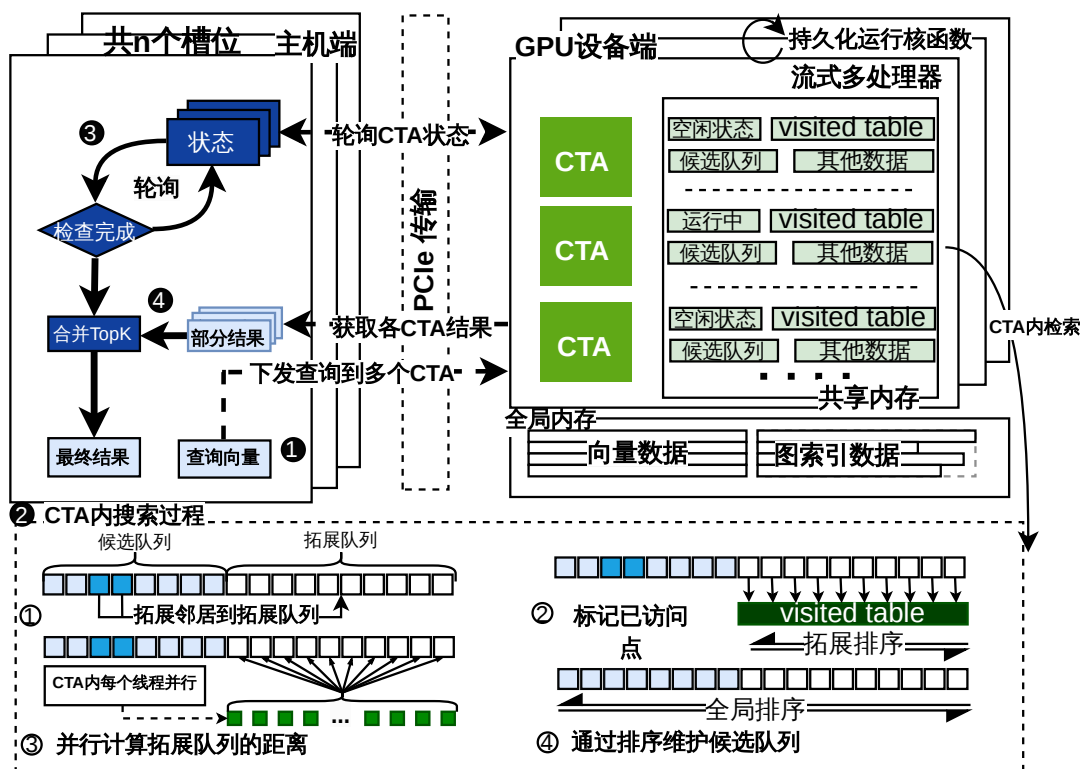


图 3.4 整体查询架构概览

第三节 GPU 动态流式处理架构

为了彻底解决批处理气泡问题，本文提出了一种 GPU 动态流式处理架构。该设计将 GPU 上传统的阻塞式同步模型转变为支持查询级异步调度模型，这一设计彻底消除了长尾查询对整体批次的阻塞效应，同时免去了为凑齐批次而产生的排队等待开销，实现了查询请求的即到即处理。

在具体实现上，我们将 GPU 的并行计算资源抽象为一组互不影响的独立槽位。每个槽位作为一个虚拟的执行单元，全权负责单个查询的完整生命周期，包括从 Host 端接收查询参数、在 GPU 端执行图搜索、以及将中间结果回传至主机。这一过程无需任何批次级别的同步操作。图 3.5 直观对比了传统批处理与动态流式处理的区别：动态流式处理允许每个查询独立运行，不受同批次内长尾查询的阻塞，从而能够更早地返回结果，同时也缩短了多批次任务的整体完成时间。

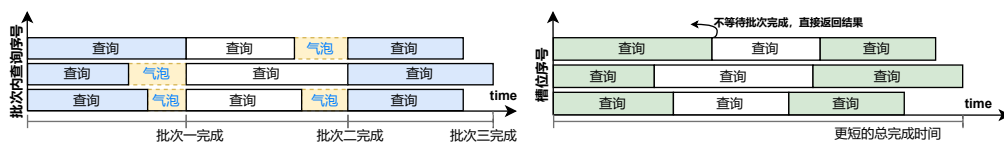


图 3.5 普通批处理 vs 动态流式处理示意图

这种设计的关键优势在于解耦：一旦某个槽位完成了当前查询的计算任务，它能够立即释放资源并请求分发下一个就绪的查询，而无需受到同一物理批次中其他未完成查询的阻塞。这种机制将同步粒度从“批次级”细化到了“查询级”。消除了多个批次调度之间的气泡，降低了平均延迟的同时，一定程度上也增加了吞吐。

一、槽位状态管理

为了实现 Host 端与 GPU 端在槽位级别的高效交互，并确保结果的及时回传，我们为每个槽位设计了一系列状态来管理其生命周期，如图 3.6 所示，包含以下五种核心状态，其流转规则如下：

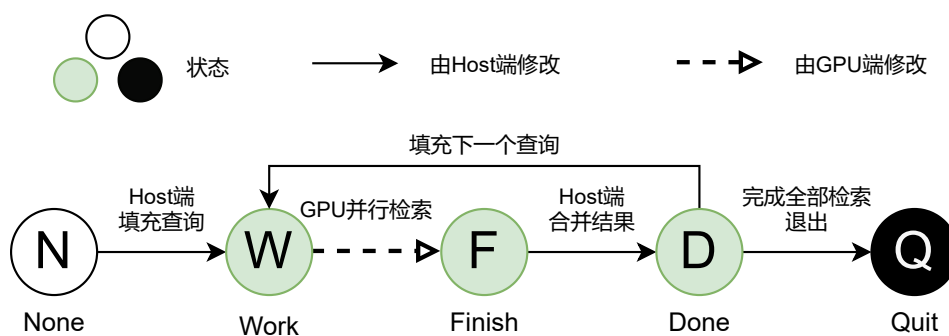


图 3.6 槽位状态流转过程

- **None(空闲)**: 槽位的初始状态，表示该槽位当前无任务，随时准备接受新的查询请求。
- **WORK(运行中)**: 当 Host 端将查询数据填充至槽位缓冲区并更新信号后，槽位状态跃迁为 WORK。此时，驻留在 GPU 上的持久化线程块们（即与槽位对应的多个 CTA）检测到该状态，开始执行具体的图搜索算法。
- **FINISH(搜索完成)**: GPU 端完成近邻搜索并将局部结果写入指定显存区域后，将自身状态标记为 FINISH，以此通知 Host 端数据已就绪。

- **DONE(结果回收):** Host 端轮询检测到槽位处于 FINISH 状态后，读取并收集查询结果。在完成数据回收后，主机将状态更新为 DONE，触发槽位的重置逻辑。
- **QUIT(退出):** 用于系统停止或资源回收。当接收到终止信号时，槽位进入 QUIT 状态，对应的 GPU 线程块退出持久化循环并释放硬件资源。

二、持久化 GPU 核函数

出于性能考虑，GPU 近邻图 ANNS 通常采用单个核函数来运行完整的一轮查询。然而，传统的单内核设计在适配动态流式处理时面临挑战，由于动态流式处理打乱了固定的批次结构，核函数无法再以简单的批次为单位下发；同时，动态流式处理要求内核具备与 Host 端灵活交互的能力，以实时检查每个槽位的状态。

一种朴素的替代方案是对单个核函数采取分段执行策略（例如基于跳数分段）。具体而言，核函数内的所有 CTA 在执行固定数量的搜索步数后强制退出，随后由 Host 端介入以检查查询状态并注入新任务。但是这种方法存在显著缺陷：首先，需要更频繁的重启核函数，这导致数据必须反复加载到共享内存中。其次，调度时机的权衡极具挑战性，频繁的检查会引入大量开销，而过疏的检查则无法及时填充空闲槽位以有效消除气泡。此外，每次内核启动均伴随着元数据初始化与参数传输等不可避免的额外开销。如图 3.7 所示，在 SIFT-1M 数据集（Top16@0.95 召回率）的测试中，当批次大小为 8 时，排除搜索时间的额外开销占比高达约 5%。

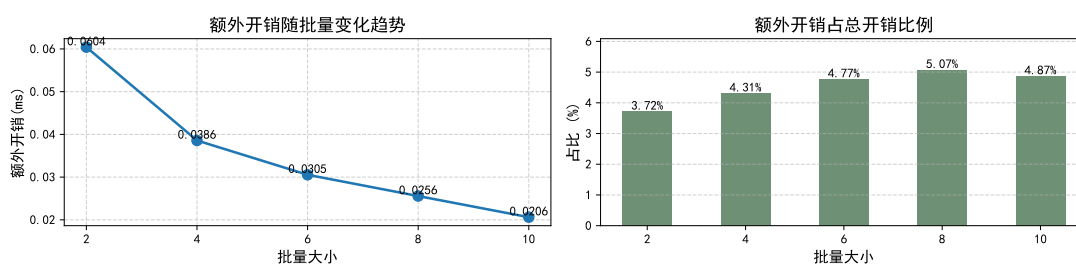


图 3.7 SIFT-1M 数据集 0.95@Top16 下，批量随额外调用开销变化

为了解决上述问题，我们设计了一种持久化核函数机制来实现 ANNS 的动态流式处理。该机制允许核函数在 GPU 上持续运行而不退出，并通过轮询模式替代传统的阻塞模式来实现状态检查。这使得核函数能够在 GPU 端自主轮询并

修改状态，而非完全依赖 Host 端的控制。

传统的阻塞式同步需要显式地将 GPU 上所有查询的状态传输回 Host 端进行判断，这种高延迟模式并不适合 ANNS 这类短时延查询。得益于 NVIDIA GPU 的统一虚拟寻址（UVM）技术，GPU 的片上存储（包括全局内存）可以被映射到 Host 端的进程地址空间。这意味着 Host 端可以通过指针直接跨 PCIe 总线访问 GPU 上的数据，而无需暂停核函数。基于此，我们实现了支持循环执行的 GPU 持久化内核，Host 端能够通过 UVM 指针轮询 GPU 状态以感知数据就绪，并直接下发新查询。

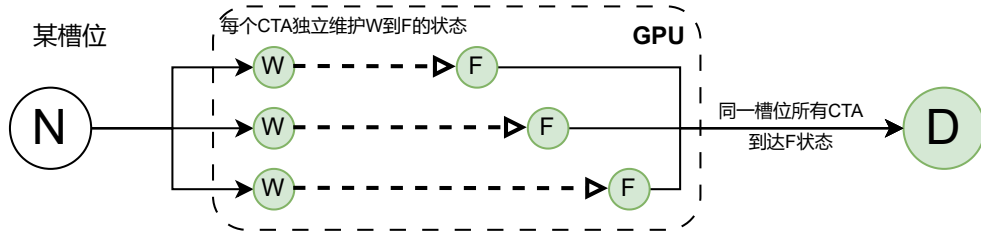


图 3.8 GPU 上状态流转过程

为了使动态流式处理架构兼容 Multi-CTA 算法，我们将槽位的状态流转细化到了 CTA 粒度。如图 3.8 所示，每个槽位将查询任务分发给多个 CTA，每个 CTA 独立维护自身的 WORK 到 FINISH 状态切换，互不干扰。Host 端轮询对应槽位下所有 CTA 的状态，当所有 CTA 均达到 FINISH 状态后，及时回收结果并注入下一个查询。在此过程中，核函数始终保持运行状态。由于同一槽位内的不同 CTA 完成检索的时间各不相同，系统允许这些 CTA 处于不同的中间状态，从而最大化并行度。

持久化核函数的设计要求所有槽位以及对应的 CTA 必须能够并发驻留 GPU，而不是被调度执行。这就要求我们在分配逻辑资源时，必须严格遵守硬件的物理限制，以确保所有槽位关联的线程块能被同时调度驻留。为了充分挖掘 GPU 的计算核心与存储资源，我们需要基于 GPU 的计算与存储约束，推导出每个槽位应分配的最优 CTA 数量 (K_{CTA})。

每个查询分配的 CTA 数量 K_{CTA} 直接决定了查询内的并行度，其数值的计算主要受限于两个核心维度的物理约束：一方面是计算资源，即单个流式多处理器（SM）能够并发调度的最大线程块数 B_{max} ；另一方面是存储资源，要求单个 SM 内所有线程块占用的共享内存总量不得超过其物理上限 M_{total} 。

首先，我们定义单个线程块的内存消耗。除了存储候选列表与扩展列表所需的共享内存 M_{use} 外，我们还需为每个块预留动态调整的 L1 缓存空间 M_{cache} 。因此，单个 SM 受限于共享内存所能支持的最大块数 B_{mem} 为：

$$B_{mem} = \lfloor M_{total} / M_{use} + M_{cache} \rfloor \quad (3.1)$$

结合硬件的物理调度限制 B_{max} ，单个 SM 实际能够并发驻留的有效块数上限 B_{limit} 取两者的最小值：

$$B_{limit} = \min(B_{max}, B_{mem}) \quad (3.2)$$

最后，基于 GPU 拥有的 SM 总数 N_{SM} ，计算全系统的最大并发承载能力，并将其均匀分配给 S_{total} 个槽位，从而得出每个查询的最优并行度 K_{CTA} ：

$$K_{CTA} = \lfloor B_{limit} \cdot N_{SM} / S_{total} \rfloor \quad (3.3)$$

通过该推导公式，系统能够在不同批处理规模（影响 S_{total} ）下，自动计算出满足硬件物理限制的最大化并行参数。

三、跨 PCIe 的状态副本

虽然轮询机制使 Host 端能灵敏地感知状态变化，但频繁跨 PCIe 总线轮询大量 CTA 的状态会引发严重的 I/O 瓶颈，进而抢占查询向量和 TopK 结果传输所需的带宽。特别是在小维度数据集场景下，计算密度的降低意味着数据交换频率的提升，使得 PCIe 瓶颈尤为突出。

本质上，Host 与 GPU 之间的状态传输是一种信息同步过程。在 Host 端高频轮询 GPU 上每个槽位的状态会生成大量微小的 PCIe I/O 事务，而在大多数情况下，状态并未发生改变，导致大量轮询成为无效操作。虽然采用阻塞模式可以减少 PCIe I/O，但其响应延迟通常不如轮询模式理想。

为了解决这一矛盾，我们提出了一种基于本地副本的状态同步机制。我们在 Host 端和 GPU 端分别为每个槽位的状态数据构建了本地副本。各端仅需轮询本地内存中的状态副本，从而避免了无效的跨 PCIe 访问。

如图 3.9 所示，我们利用 UVM 技术将 Host 和 GPU 上的状态副本地址相互映射到对方的寻址空间。当 GPU 需要更新槽位状态时（例如从 WORK 变为 FINISH），它首先更新本地状态，随后发起一次 PCIe 写操作来更新 Host 端的远程副本。同理，Host 端在下发查询时也执行类似操作。

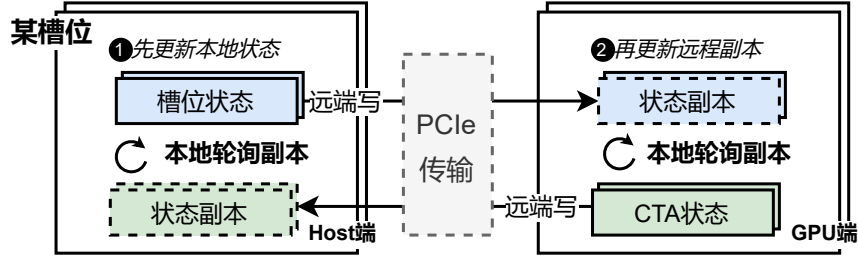


图 3.9 副本同步状态流程

通过这种设计，在状态未发生变化的轮询期间，系统不会产生任何 PCIe 流量。尽管 GPU 和 Host 均可修改状态，但由于状态机的流转规则严格限制了在任一时刻仅有一方拥有修改权（例如 GPU 仅在收到查询后拥有从 WORK 到 FINISH 的修改权，而 Host 保留其他状态的修改权），因此不会出现数据一致性冲突。

四、完整检索流程

为了深入阐述系统的运行机制，本小节描述了查询执行的具体流程。假设查询目标数量为 K ，候选列表的最大容量为 L ，阶段切换阈值为 O ，系统共分配 T 个 CTA 协同执行搜索。各步骤与总体架构图 3.4 中的黑色序号对应。

1. **查询分发与初始化：**CPU 线程将查询请求写入指定槽位，并将该槽位关联 CTA 的状态标志置为 *WORK*。驻留的 CTA 持续轮询状态，一旦检测到 *WORK* 信号，即刻启动搜索流程。同时，各 CTA 协同初始化基于位图实现的全局已访问表。
2. **CTA 内并行搜索：**每个 CTA 独立执行如 **算法 2** 所示的搜索逻辑，这部分将在 第四节 一节中解释。该过程主要包含四个核心流程：
 - **候选点扩展：**CTA 从候选列表中选取最近的未访问节点。若当前节点在列表中的偏移量超过阈值 O （进入收敛阶段），则启用束扩展策略，同时选取后续多个候选点进行批量扩展；否则仅选取队首节点（定位阶段）。所有选中节点的邻居被加入扩展列表。若无法获取有效的未访问节点，则判定收敛，将状态置为 *FINISH*。
 - **去重过滤：**利用共享位图并发检查扩展列表，过滤掉历史已访问节点，并原子标记新访问的节点。
 - **距离计算：**计算查询向量与扩展列表中新节点的距离。利用 CTA

内的线程级并行，将向量维度分配给不同线程计算，并通过 Warp Shuffle 指令进行寄存器级规约。

- **列表维护：**执行双调排序网络，对扩展列表按距离排序。随后，将有序的扩展列表与原候选列表进行归并，保留前 L 个最优节点 ($L \geq K$) 以更新候选列表。

3. **Host 端状态监控：**Host 端持续轮询当前槽位关联的所有 CTA 状态。一旦检测到所有 T 个 CTA 均进入 *FINISH* 状态，即判定 GPU 端搜索结束。Host 随即从显存读取各 CTA 的局部结果，并重置槽位状态以加载下一个查询。
4. **结果合并与输出：**Host 端利用优先级队列对收集到的 T 个局部结果集进行多路归并，筛选出全局 Top-K 结果，在过滤掉可能存在的冗余数据后提交最终响应。

第四节 并行检索算法优化设计

本节将详细阐述针对 Multi-CTA 架构执行层瓶颈的优化方案。首先，针对 CTA 内部排序开销过高的问题，本文提出了一种分阶段束搜索策略；其次，针对 CTA 之间结果合并的同步与访存瓶颈，本文设计了一套 CPU-GPU 异构协同流水线，通过卸载 TopK 合并任务来提升整体吞吐量。

一、分阶段束搜索策略

为了进一步提升 GPU 在向量检索中的计算密度，降低非计算指令的占比至关重要。如第三章研究动机所述，CTA 内部的候选列表的维护是主要的性能瓶颈。传统的基于近邻图的 ANNS 算法普遍遵循严格的贪婪策略：即在每次迭代中，必须精确筛选出距离查询点最近的未访问节点作为下一跳，并计算其邻居距离。随后，新生成的邻居节点必须被插入候选列表并重新排序，以维护列表的有序性。这种“一节点一更新”的细粒度维护逻辑虽然确保了搜索路径的精确性，但在 GPU 并行架构下，随着距离计算被高度并行化加速，排序操作的开销占比反而凸显，成为制约延迟的关键因素。

GPU 上高昂的排序开销促使我们重新审视在每次迭代中维持严格有序的重要性。通过收集并分析查询过程中“查询向量与候选点距离”的变化数据（如图 3.10 所示），我们观察到了显著的距离收敛特性，整个搜索过程呈现出阶段性特

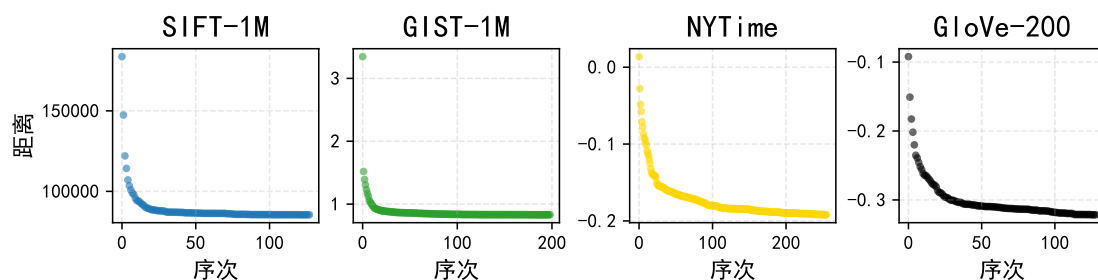


图 3.10 不同数据集下查询距离收敛的特征

征。在搜索初期，候选点与查询向量的距离呈现急剧下降趋势。这表明算法正处于快速导航阶段，致力于从图的边缘向包含最终 TopK 结果的目标邻域逼近。随着迭代进行，距离下降曲线趋于平缓并逐渐收敛。这表明搜索已进入目标邻域内部，此时的操作转变为在局部小区域内的扩散与精细化查找。

基于上述观察，我们认为在收敛阶段维持高频的严格排序存在冗余，可以适当松弛。因为一旦搜索进入了包含 TopK 的局部小区域（搜索后期），为了保证高召回率（尤其是针对 TopK 较大的场景），算法往往需要遍历该区域内的大部分节点。在这种情况下，访问的先后顺序对最终结果的影响微乎其微——既然这些点最终大概率都会被访问，那么在局部范围内严格遵循贪婪顺序通过排序来换取路径优化的收益极低。

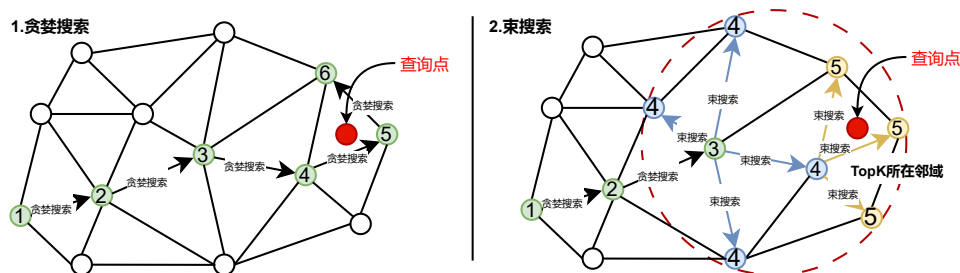


图 3.11 分阶段束搜索

基于此，我们提出了一种分阶段束搜索方法，旨在根据搜索的进展动态调整排序策略，如图3.11所示。具体而言，我们引入了一种基于候选列表访问深度的自适应触发器。系统实时监控当前被选中进行扩展的候选节点在列表中的偏移量，一旦偏移量超过预设阈值，即判定搜索已从“快速定位”转入“局部查找”，从而触发阶段切换。在不同的阶段，算法执行不同的搜索模式：

- **定位阶段：**在早期，为了确保导航方向的准确性，我们沿用标准的贪婪搜

索模式，在每次迭代后执行严格的列表维护，从而快速锁定目标区域。

- **收敛阶段：**进入后期，算法切换至束搜索模式。在此模式下，系统不再每次仅选择一个最优节点，而是一次性并发选择多个候选点进行邻域扩展。同时，我们采用延迟排序策略，允许新生成的邻居节点先暂存于缓冲区，跳过若干次排序操作，仅在累积一定数量后进行一次统一的合并与维护。值得一提的是，束搜索策略在高召回率下（长候选队列）表现明显，这是因为高召回率下有着更长的收敛阶段，需要探索更大范围的小区域。

为了客观评估分阶段束搜索的性能优势，我们建立了搜索开销模型。设 GPU 并行线程数为 P ，GPU 上的排序操作使用并行双调排序^[40]，单位并行比较-交换操作的基准开销为 τ （这是双调排序中的一个基本操作）。

双调排序网络由若干双调合并阶段组成。对于长度为 N 的序列：

1. **双调合并** $T_{merge}(N)$ ：网络深度为 $\log N$ 。受限于并行度 P ，其开销为：

$$T_{merge}(N) \approx \frac{N}{2P} \cdot \log N \cdot \tau \quad (3.4)$$

2. **双调排序** $T_{sort}(N)$ ：由 $\log N$ 次合并构成，网络总深度约为 $\frac{1}{2} \log^2 N$ 。其开销为：

$$T_{sort}(N) \approx \frac{N}{2P} \cdot \frac{\log^2 N}{2} \cdot \tau \quad (3.5)$$

在实际工程实现中，为了适配双调网络，数据规模通常需填充至最近的 2 的幂次。由于候选列表 L （如 128, 256, 512）通常远大于邻居数 D （如 8, 16, 32），列表合并操作的实际处理规模 N 通常由 L 的上取整幂次决定。为简化分析，我们记合并规模为 $2L$ （即 $N \approx 2L$ ，包含候选队列和排序后的邻居队列）。

在贪婪搜索模式下，算法需执行 W 次迭代（ W 为束宽度）。每次迭代需对 D 个新邻居排序，并将其合并入 $2L$ 规模的列表（候选队列本身已经有序），同时需执行一次节点查找。总开销 $Cost_{greedy}$ 为：

$$Cost_{greedy} = W \cdot [T_{sort}(D) + T_{merge}(2L) + \Delta_{find}] \quad (3.6)$$

在束搜索模式下，算法将 W 次扩展合并。虽然邻居排序规模增加至 $W \cdot D$ ，但列表合并与节点查找仅需执行一次。总开销 $Cost_{beam}$ 为：

$$Cost_{beam} = T_{sort}(WD) + 1 \cdot T_{merge}(2L) + 1 \cdot \Delta_{find} \quad (3.7)$$

将两式比较可发现，束搜索的核心收益来源于节省了 $W - 1$ 次高昂的列表合并与节点查找开销：

$$(W - 1) \cdot (T_{merge}(2L) + \Delta_{find}) > T_{sort}(W \cdot D) - W \cdot T_{sort}(D) \quad (3.8)$$

为了量化证明其优势，我们定义收益比 R ，即“节省的合并开销”与“增加的排序开销”之比。束搜索性能更优的条件是 $R > 1$ ：

$$R = \frac{(W - 1) \cdot T_{merge}(2L)}{T_{sort}(WD) - W \cdot T_{sort}(D)} > 1 \quad (3.9)$$

为了直观评估该比值，我们将前述的时间复杂度公式代入并化简可得：

$$R \approx \frac{2(W - 1) \cdot L \log L}{W \log W \cdot D \log D} \quad (3.10)$$

R 的核心取决于 $L \log L$ 与 $D \log D$ 的相对量级。在实际应用中，候选列表 L 通常比邻居数 D 大一个约数数量级。例如，当 $L = 128, D = 32, W = 2$ 时，分子项 $L \log L$ 远大于分母中的 D 相关项，计算所得比值 $R \gg 1$ 。可以认为，虽然束搜索引入了的额外排序计算量，但其收益（节省的 $W - 1$ 次大规模列表合并）占据主导地位。此外，该模型尚未计入束搜索节省节点查找 $((W - 1) \cdot \Delta_{find})$ 开销，实际的性能收益比将高于理论保守估计值。

算法 2 形式化地描述了该束搜索的具体过程。

第四章的实验结果表明，这种分阶段策略虽然在后期轻微降低了搜索的贪婪度，但得益于局部区域的全面覆盖特性，其对最终召回率的影响可忽略不计。更重要的是，该方法大幅减少了排序调用次数，显著提升了 GPU 的指令执行效率，从而直接降低了端到端的查询延迟。

二、CPU-GPU 异构协同流水线

正如前文所述，Multi-CTA 算法虽然有效降低搜索延迟，但其结果聚合阶段引入了新的操作。在纯 GPU 实现中，因为同槽位多个 CTA 可能分布在不同 SM 上，将多个 CTA 的局部 TopK 结果合并为全局 TopK 需要访问高延迟的全局显存来同步结果，降低了硬件资源的利用率。此外，这种基于 GPU 内部的合并机制与本文提出的流式处理架构兼容性较差，无形中增加性能开销。

我们通过分析整个查询流程发现，TopK 合并是 GPU 上单个查询生命周期的最后一个操作。这意味着该步骤的输出即为最终结果，无需再次回传至 GPU 进行后续计算。因为结果最终必须返回 Host 端，我们提出了一种异构流水线策略：将计算密集型的图搜索保留在 GPU 上，而将逻辑复杂但数据量较小的 TopK 合并操作卸载至 CPU 执行。

Algorithm 2 CTA 内分阶段束搜索**Require:** 查询向量 q , 图索引 G , 候选列表容量 L , 阶段切换阈值 O **Ensure:** 局部 Top- K 候选结果 $\mathcal{C}$

```

1: 初始化包含入口点  $E$  的候选列表  $\mathcal{C}$ 
2: 初始化局部扩展列表  $\mathcal{E} \leftarrow \emptyset$ 
3: 协同初始化全局已访问位图  $\mathcal{V}$ 
4: while True do
5:   /* 1. 候选点选择与扩展 */
6:    $u \leftarrow$  获取  $\mathcal{C}$  中第一个未访问节点
7:   if  $u$  为 NULL then
8:     break {收敛: 无更多未访问候选点}
9:   end if
10:  在  $\mathcal{C}$  中标记  $u$  为已访问
11:  将邻居  $\mathcal{N}(u)$  加入  $\mathcal{E}$ 
12:   $offset \leftarrow$  获取  $u$  在  $\mathcal{C}$  中的索引
13:  if  $offset > O$  then
14:     $u_{next} \leftarrow$  获取  $\mathcal{C}$  中  $u$  之后的更多未访问节点
15:    if  $u_{next}$  不为 NULL then
16:      在  $\mathcal{C}$  中标记  $u_{next}$  为已访问
17:      将邻居  $\mathcal{N}(u_{next})$  加入  $\mathcal{E}$ 
18:    end if
19:  end if
20:  /* 2. 基于共享位图的过滤 */
21:  for all  $\mathcal{E}$  中的节点  $v$  in parallel do
22:    if BitmapCheckAndSet( $\mathcal{V}, v$ ) then
23:      从  $\mathcal{E}$  中移除  $v$  {已被其他 CTA 访问}
24:    end if
25:  end for
26:  /* 3. 并行距离计算 */
27:  for all  $\mathcal{E}$  中的节点  $v$  in parallel do
28:     $d \leftarrow \text{ComputeDistance}(q, v)$  {Warp 级归约计算}
29:    更新  $v$  的距离为  $d$ 
30:  end for
31:  /* 4. 列表维护 (双调排序) */
32:  BitonicSort( $\mathcal{E}$ ) {按距离对扩展列表排序}
33:   $\mathcal{C} \leftarrow \text{MergeAndTruncate}(\mathcal{C}, \mathcal{E}, L)$  {合并并保留前  $L$  个}
34:  清空  $\mathcal{E}$ 
35: end while
36: return  $\mathcal{C}$ 

```

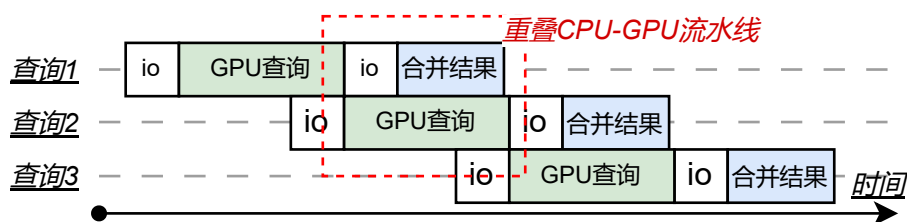


图 3.12 CPU-GPU 协同处理流水线示意图

如图 3.12 所示，这种设计使得 GPU 专注于其擅长的低通信、高并行的向量距离计算与图遍历；而 CPU 则利用其低延迟的内存访问能力和乱序执行优势，高效处理 TopK 的归并排序。在这一架构下，GPU 上的各 CTA 仅需维护共享的已访问节点表以避免重复搜索，彻底消除了 CTA 之间全局同步，并且重叠了合并开销，使得吞吐量进一步提升。

此外，这种 CPU-GPU 协同设计与前述的动态流式处理架构形成了完美的互补。通过将合并操作剥离，我们为 GPU 确立了单一的搜索职责，减少了各 CTA 的交互需要；CPU 可以在 Host 端为每个查询槽位独立、异步地执行 TopK 合并任务，无需等待整个批次的同步信号。这种 GPU 专注搜索，CPU 异步合并的协同流水线设计，有效掩盖了结果回传与处理的延迟，提升了系统的端到端吞吐量。

表 3.1 不同规模下的两种合并方式耗时对比

召回数	CTA 数量	总数	GPU 合并后回传	回传后 CPU 合并
16	16	256	0.037074 ms	0.015824 ms
32	16	512	0.036996 ms	0.018002 ms
64	16	1024	0.032541 ms	0.022396 ms
128	16	2048	0.033657 ms	0.035813 ms
16	32	512	0.031584 ms	0.017311 ms
32	32	1024	0.032397 ms	0.022112 ms
64	32	2048	0.032831 ms	0.032621 ms
128	32	4096	0.042683 ms	0.063605 ms
16	64	1024	0.032650 ms	0.022379 ms
32	64	2048	0.032795 ms	0.032456 ms
64	64	4096	0.042612 ms	0.055509 ms
128	64	8192	0.051019 ms	0.114814 ms

为了验证上述策略的有效性，我们对 TopK 合并阶段的开销进行了基准测试。如表 3.1 所示，我们对比了不同 CTA 数量和召回规模下，GPU 片上合并与 CPU 卸载合并的耗时。实验数据表明当待合并元素总数较少（例如小于 2048 个）时，CPU 合并（回传后 CPU 合并）的耗时低于 GPU 合并。这是因为 GPU 在处理小规模任务时，其内核启动开销（Kernel Launch Overhead）和全局显存访问延迟占据了主导地位，而 CPU 凭借极低的各种启动延迟和高效的缓存体系，在处理轻量级排序任务时更具优势。

此外，随着召回数 K 和 CTA 数量的增加（如表中红色数据所示），待排序元素增多，CPU 的计算耗时开始略微超过 GPU。然而，在实际的 ANNS 检索场景中，图搜索的核心计算（距离计算与图遍历）通常耗时在毫秒级，远大于微秒级的 TopK 合并开销。通过异构流水线设计，CPU 执行合并操作的时间完全可以被 GPU 处理下一个查询批次的搜索时间所覆盖（Overlap）。因此，即便在大数据量下 CPU 合并略慢，只要其耗时小于 GPU 的搜索耗时，这种开销就不会阻塞流水线，从而保证了系统整体的高吞吐量。

三、Host 端多线程与 IO 优化

在查询运行期间，Host 端需要负责查询的下发、状态轮询以及结果的获取与合并。在前文中提到，为了兼顾吞吐量，系统设计支持多个槽位并发运行。由于采用了动态流式处理架构，每个查询的返回时机是不确定的，CPU 需要不断轮询这些查询对应 CTA 的状态，并通过 PCIe 总线取回结果。当槽位数量较多，特别是在处理低维或小规模数据集（计算耗时短、返回频率高）时，单个 CPU 线程往往难以通过串行轮询及时响应如此高频的 IO 请求，导致 Host 端成为新的瓶颈。

为了应对这一问题，我们在 Host 端引入了多线程并发来处理。具体而言，我们将所有槽位划分为若干组，每组分配一个独立的 CPU 线程负责管理。此外，我们为每个 CPU 线程分配了私有的 CUDA 流。在实现上，属于同一个 CPU 线程的槽位共享该私有流，所有的查询下发和结果回传均通过该流异步传输。这种设计不仅避免了多线程争抢单一 IO 流导致的互斥锁开销，还充分利用了 PCIe 的全双工带宽。

第五节 本章小节

本章详细介绍了面向低延迟的高性能 GPU 向量检索系统的设计与实现。首先，针对调度层的批处理气泡问题，提出了基于多槽位的动态流式处理架构，结合持久化核函数与 PCIe 状态副本技术，实现了查询级的细粒度调度与零开销任务分发。其次，针对执行层的效率瓶颈，设计了分阶段束搜索算法与 CPU-GPU 异构协同流水线，通过自适应降低排序频率和卸载合并任务，显著提升了资源利用率。这一系列软硬协同的优化措施，共同确保了系统在维持高吞吐量的同时，能够满足严苛的在线低延迟需求。

第四章 实验设计与分析

第一节 实验环境

本节详细介绍了用于评估本系统架构检索性能的实验环境与实验数据。

一、实验配置

所有实验均在搭载 Ubuntu 20.04.1 操作系统（Linux 内核版本 5.15.0）的高性能服务器上进行。该服务器采用双路 NUMA 架构，包含两个节点，每个节点配备一颗 Intel Xeon Silver 4210 处理器（主频 2.20GHz，拥有 10 个物理核心）以及 128GB DRAM，系统总内存容量为 256GB。

表 4.1 NVIDIA RTX A6000 硬件参数详情

硬件属性	数值
架构代号 (Architecture)	Ampere
计算能力 (Compute Capability)	8.6
流式多处理器数量	84
全局内存 (Global Memory)	47.32 GB
L2 缓存大小	6.0 MB
每线程块共享内存 (Shared Mem / Block)	48 KB
每 SM 共享内存 (Shared Mem / SM)	100 KB
每块寄存器数量 (Registers / Block)	65536
每块最大线程数 (Max Threads / Block)	1024
线程束大小 (Warp Size)	32
实测全局内存带宽	682.84 GB/s
实测共享内存带宽	75000.00 GB/s

为了实现高效的并行计算与加速，服务器配备了一张 NVIDIA RTX A6000 GPU^[41]，驱动版本为 535.104.05。该 GPU 基于 NVIDIA Ampere 架构，计算能力为 8.6。它拥有 84 个流式多处理器，配备了 48GB 的 GDDR6 全局显存以及 6MB 的 L2 缓存。关于 GPU 的详细硬件参数如表 4.1 所示。

二、实验数据集

为了全面且客观地评估所提检索系统的性能，本文选取了四个在 ANNS 研究中被广泛采用的公开基准数据集。这些数据集涵盖了**计算机视觉与自然语言处理**等核心领域，且在向量维度、数据规模及度量空间上呈现出多样性与区分度，能够充分验证算法在不同数据分布下的鲁棒性。

表 4.2 详细列出了各数据集的统计特征。每个数据集均由三部分组成：用于构建索引的底库向量集（Base Set）、用于测试的查询向量集（Query Set）以及基于暴力搜索计算得到的标准 Top-100 真值集（Ground Truth）。

表 4.2 实验的数据集

数据集	数据量	维度	距离类型	所属领域
SIFT-1M ^[42]	1,000,000	128	Euclidean	计算机视觉
GIST-1M ^[42]	1,000,000	960	Euclidean	计算机视觉
GLoVe200 ^[43]	1,183,514	200	Cosine	自然语言处理
NYTimes ^[44]	290,000	256	Cosine	自然语言处理

第二节 综合性能对比实验

为了全面衡量本系统（命名为 **ALGAS**）的性能指标，我们在当前基于 GPU 的近邻图 ANNS 领域选取了 **GANNS** 和 **CAGRA** 作为基线工作进行对比。

实验主要关注系统每秒吞吐量和查询延迟两个核心维度。为了保证对比的公平性，我们将 **ALGAS** 的并发槽位数量设定为与基线系统的批次大小一致，以确保系统处于相同的负载并发度下。

一、实验设置

由于 **CAGRA** 与 **GANNS** 两个基线工作分别设计了针对 GPU 优化的图索引，且其检索算法均与其原生索引高度适配。为剥离图拓扑结构差异对性能的影响，专注于评估搜索算法本身的效率，我们在实验中控制了图索引变量。在评估 **ALGAS** 时，分别直接复用上述两个基线生成的图索引。这种设计既确保了基线算法能发挥其固有结构优势，也验证了 **ALGAS** 对不同图结构的广泛适应性。

其中，**CAGRA** 索引紧贴 GPU 硬件特性，采用固定出度设计，追求极致的构建速度与访存友好性；而 **GANNS** 版 NSW 索引则兼顾小世界特性与并行效

率，对节点度数设置了动态的上下限约束。相关构图参数详见表 4.3。为确保搜索阶段输入的一致性，我们对所有索引进行了统一的存储格式预处理。此外，所有实验结果均为 GPU 充分预热后测得。

表 4.3 不同图索引在各数据集上的构图参数配置

图索引类型	数据集	构图探索深度	邻居数目	距离类型
NSW (GANNNS)	SIFT-1M	128	32	Euclidean
	GIST-1M	64	32	Euclidean
	NYTimes	128	64	(归一化) Inner Product
	GLoVe200	128	64	(归一化) Inner Product
CAGRA	SIFT-1M	64	32	Euclidean
	GIST-1M	64	32	Euclidean
	NYTimes	128	32	(归一化) Inner Product
	GLoVe200	128	64	(归一化) Inner Product

二、不同召回率下综合性能比较

图 4.1 和图 4.2 分别展示了在 Top-16 检索、不同索引类型下，各系统吞吐量和平均延迟随召回率变化的性能曲线。纵轴在不同的图中分别表示每秒吞吐量和查询的平均延迟，横轴均表示检索结果的召回率；左侧一列为 CAGRA 索引的测试结果，右侧为 GANNNS 版-NSW 索引的测试结果。由于不同检索方法和图索引本身结构的原因，存在召回率的上下限，部分曲线的召回率区间未能完全覆盖。例如，ALGAS 在 SIFT-1M 这类小型数据集上性能曲线已趋于饱和。

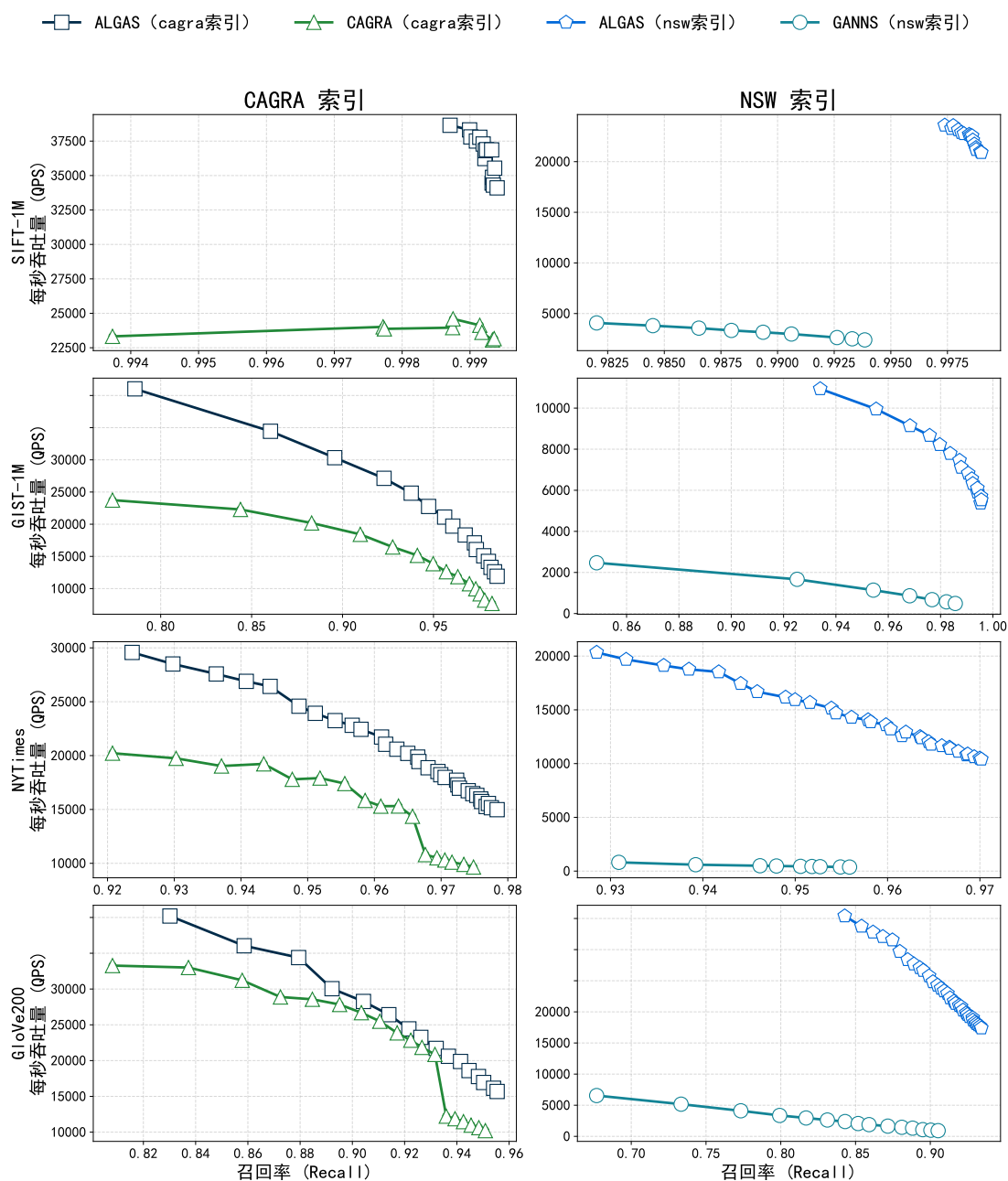


图 4.1 不同数据集与图索引下，吞吐量 (QPS) 随召回率 (Recall) 的变化趋势（越靠近右上方性能越优）。

从图 4.1 可以看出，ALGAS 的性能曲线显著位于基线工作的右上方，这表明在相同的召回率水平下，ALGAS 能够提供更高的吞吐量。相应地，在图 4.2 中，ALGAS 的曲线位于更靠右下方的区域，意味着在同等精度下其延迟更低。

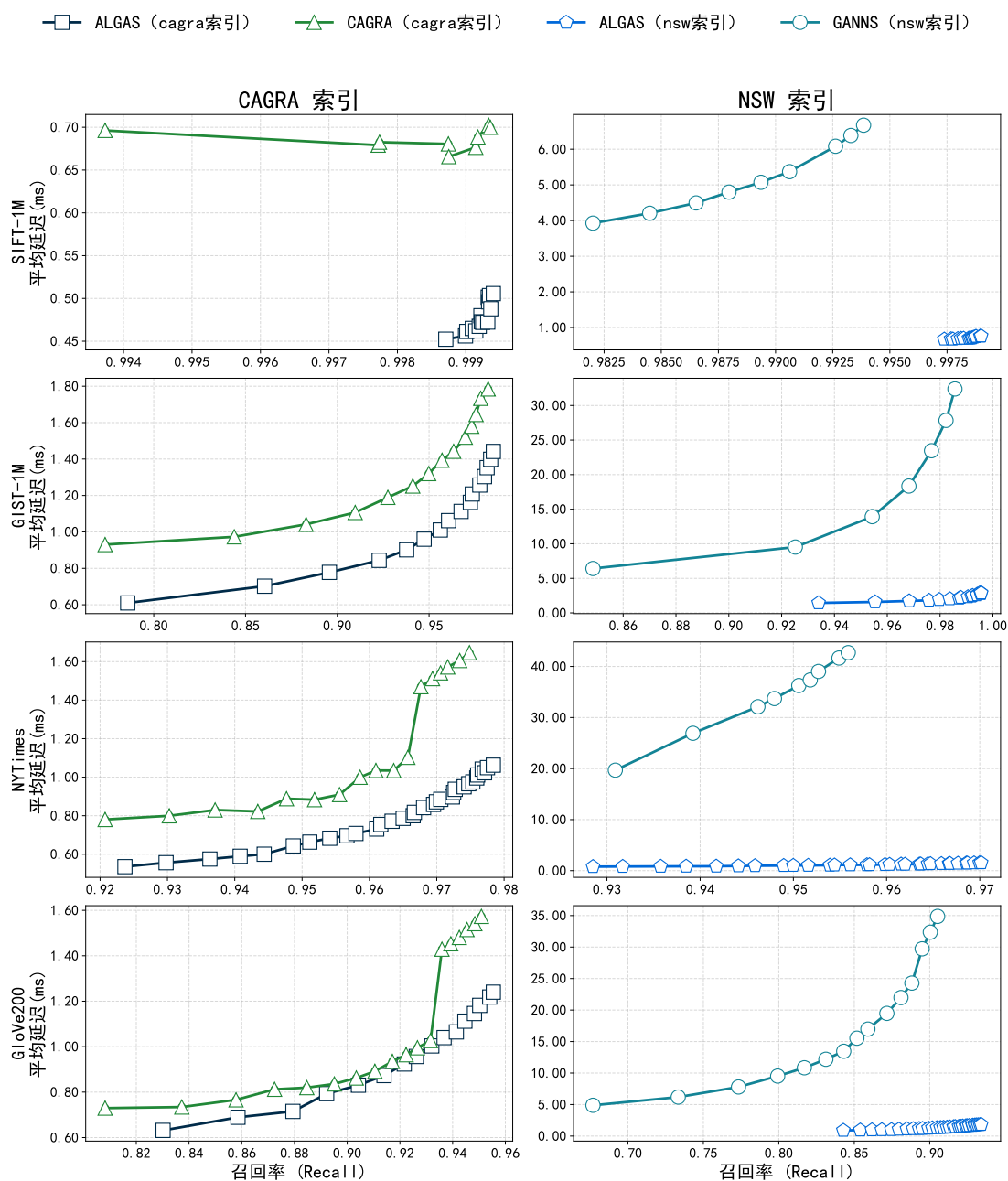


图 4.2 不同数据集与图索引下，平均延迟 (Latency) 随召回率 (Recall) 的变化趋势（越靠近右下方性能越优）。

定量分析数据显示，ALGAS 在所有测试数据集和图索引类型上均展现了卓越的性能。与当前最先进的基线工作 CAGRA 相比，ALGAS 平均查询延迟降低了 21.9% 至 35.4%，每秒吞吐量提高了 27.8% 至 55.2%。

特别值得注意的是，GANNS 在实验中表现出明显的性能瓶颈。根本原因在

于 GANNS 缺乏对 Multi-CTA 并行搜索的支持，导致其在小批量处理场景下无法有效填充 GPU 的海量计算核心，造成严重的硬件资源闲置。这种并行度不足的问题在许多未针对小批量场景优化的 GPU 图算法中普遍存在。

三、不同批次下综合性能比较

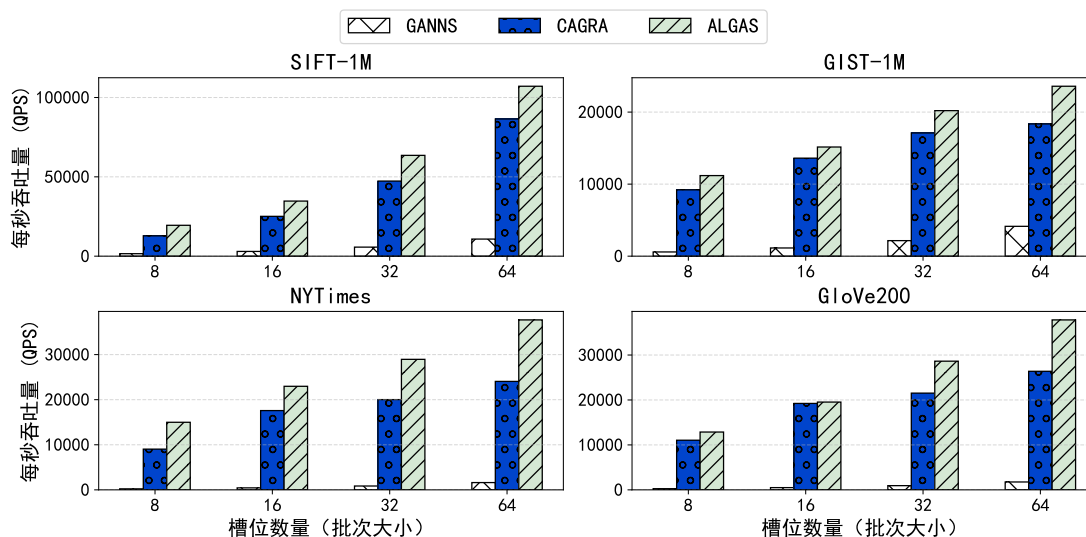


图 4.3 不同数据集下，各系统每秒吞吐量随批次大小的变化趋势

并发槽位数量（在其他基线工作中对应为批次大小）是平衡系统吞吐量与查询延迟的关键参数。该参数的选择面临着典型的权衡：一方面，增加槽位数可以提升查询间并行度，提升整体吞吐量；但另一方面，过多的并发查询会加剧对 GPU 共享内存和计算资源的争抢，限制了每个查询可分配的 CTA 数量，反而可能增加单个查询的处理延迟。此外，较大的批次还意味着主机与设备之间更频繁且大规模的数据交换（如查询分发与结果收集），从而增加了 PCIe 总线的传输压力。

图 4.3 和图 4.4 分别展示了在固定召回率约束下，各系统性能随槽位（批次）大小（8, 16, 32, 64）的变化情况。

实验结果表明，ALGAS 在所有批次设置下均保持了最高的吞吐量。与强基线 CAGRA 相比，ALGAS 的 QPS 提升了 **18.8% 至 145.9%**。特别是在槽位达到 64 的情况下，ALGAS 的吞吐量优势尤为明显（如图 4.3 所示）。

在延迟表现上，ALGAS 始终优于 CAGRA 和 GANNS（如图 4.4 所示）。即便在较大批次（如 64）下，ALGAS 依然能有效控制排队延迟，相比 CAGRA 降

低了约 **17.7% 至 61.8%** 的查询延迟。而 GANNS 在小批次下表现与 ALGAS 和 CAGRA 有明显差距。

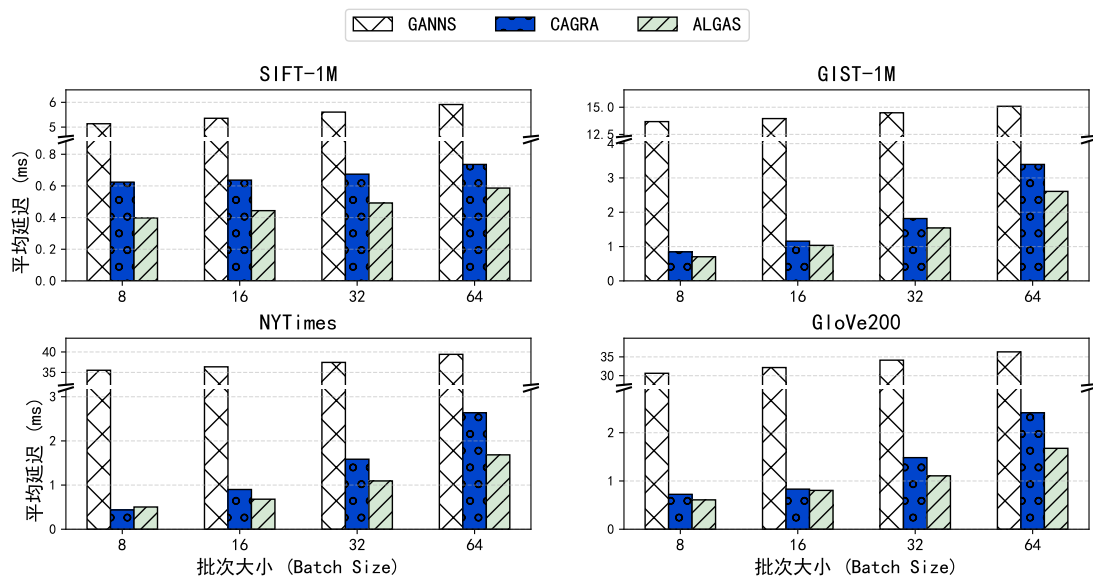


图 4.4 不同数据集下，各系统平均查询延迟随批次大小的变化趋势

四、Top-K 召回规模对性能的影响

为了评估系统在不同召回量下的性能，我们测试了 ALGAS 在不同 K 值下的性能表现。图 4.5 展示了 SIFT-1M 和 GIST-1M 数据集上，平均延迟随 Top-K (20 ~ 60) 的变化趋势，图中红色数值标注了对应的实际召回率 (Recall)。

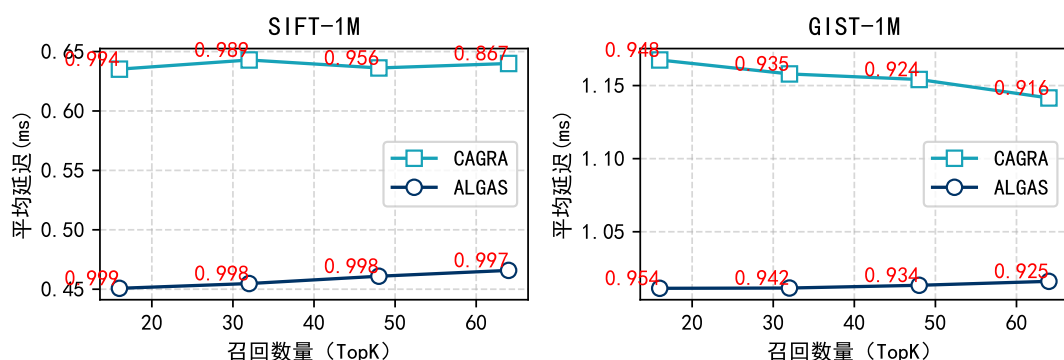


图 4.5 不同 Top-K 规模下，ALGAS 与 CAGRA 的平均延迟对比及召回率（红色）变化

可以发现，ALGAS 的延迟曲线（深蓝色圆形）相较于 CAGRA（浅蓝色方形）更为平缓且始终保持在较低水平。例如在 SIFT-1M 上，ALGAS 将延迟稳定

在 0.46ms 至 0.50ms 之间，而 CAGRA 则在 0.6ms 以上。

在维持低延迟的同时，随着 Top-K 值的增加，两者的召回率都有一定损失，但 ALGAS 依然保证了极高的召回精度。如图中红色标注所示，在 SIFT-1M 数据集上，ALGAS 在所有 K 值下均保持了接近 0.998 的超高召回率；在 GIST-1M 上，其召回率也稳定在 0.93 左右，证明了分阶段束搜索策略在修剪计算量的同时并未牺牲检索质量。

第三节 消融实验与结果分析

一、动态流式处理架构的有效性分析

为了深入验证动态流式处理架构相对于静态普通批处理在延迟优化方面的优势，我们在 SIFT-1M、GIST-1M、NYTimes 和 GloVe200 四个数据集上进行了对比测试。我们收集了测试过程中所有查询请求的端到端延迟数据，并按耗时从小到大排序，绘制了延迟分布曲线，如图 4.6 所示。图中绿色曲线代表普通批处理，蓝色曲线代表动态流式处理，虚线标识了 P25、P50（中位数）和 P75 的延迟分位点。

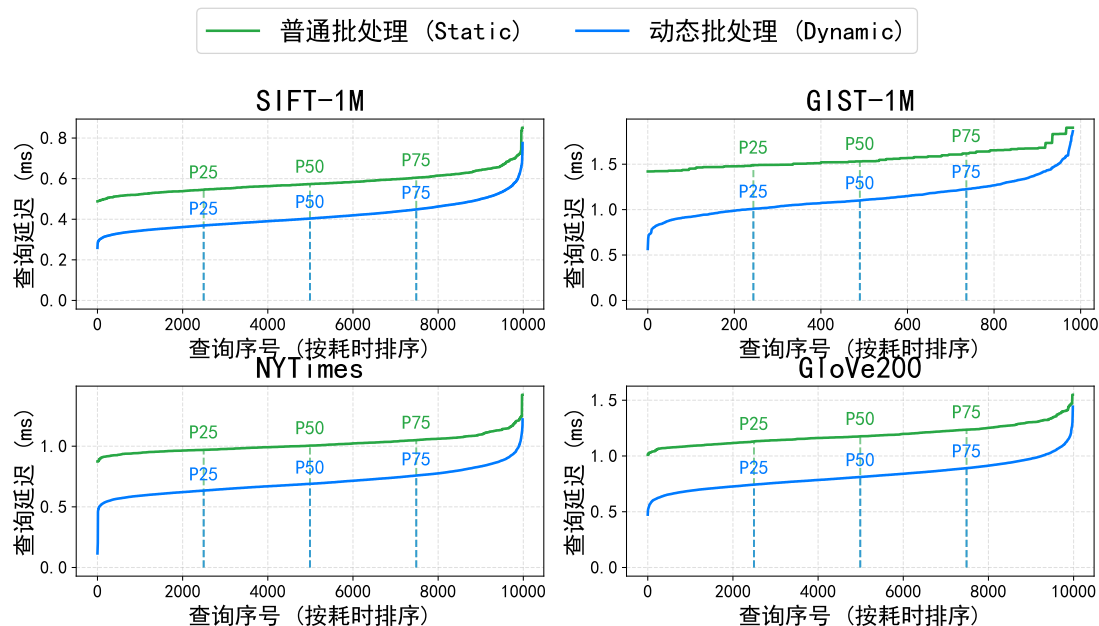


图 4.6 不同数据集下静态批处理与动态流式处理的查询延迟分布对比

从图 4.6 的实验结果可以清晰地观察到，在所有四个数据集上，动态流式处理的延迟曲线（蓝色）始终显著位于普通批处理曲线（绿色）的下方，这表明在

同等查询负载下，动态流式处理能够全方位地降低响应时间。具体而言，普通批处理的曲线呈现出较高的起始延迟且整体较为平缓，即整个批次的返回时机受限于批次中处理最慢的那个查询，导致即使是计算量极小的简单查询也被迫空转等待整个批次完成。相比之下，动态流式处理通过细粒度的任务调度不需要批处理中的同步等待，允许查询在完成计算后立即从 GPU 返回 Host 端。这种解耦机制使得 P25、P50 和 P75 等分位点的性能均得到显著提升，拥有更低的端到端延迟，特别是在 SIFT-1M 和 GIST-1M 数据集上，中位延迟大幅降低。这不仅消除了不必要的资源等待，极大地降低了系统的平均延迟，从而显著提升了整体吞吐效率。

二、多线程并发与状态副本机制的有效性分析

在数据交互频繁的情况下，Host 端的调度效率往往是制约整体性能的关键因素之一。为了探究 Host 端并行策略及状态同步机制的影响，我们评估了系统在不同线程配置及不同状态同步模式下的性能表现。首先，我们验证了 Host 端多线程并行处理的必要性。如图 4.7 (A) 和 (B) 所示，随着 Host 线程数的增加（从 1 增加至 6），系统的每秒吞吐量（QPS）呈现显著上升趋势，同时平均查询延迟明显下降。这表明多线程设计能够有效掩盖 CPU 端的任务分发延迟与结果聚合开销，充分利用 PCIe 带宽与 GPU 的计算能力。

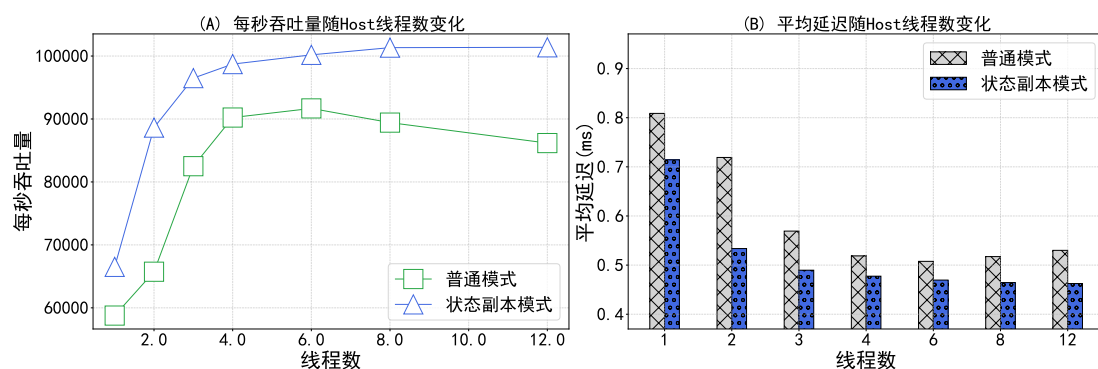


图 4.7 不同主机线程数下，普通模式与状态副本模式的吞吐量 (A) 与延迟 (B) 对比

对于 SIFT-1M 这类低维度（128 维）数据集，由于单次查询的数据传输量较小，Host 端的调度频率极高，因此极易受限于 IO 瓶颈。图 4.7 的对比结果展现了状态副本模式的优势。在普通模式下，当线程数超过 8 之后，吞吐量的拓展性不足（图 A 中绿色曲线尾部下滑），这表明频繁的跨设备状态轮询受限于的 PCIe 总线竞争。相比之下，状态副本模式（蓝色曲线）表现出了可扩展性，即

便在 12 线程的并发下依然保持了平稳的高吞吐量，且在所有线程设置下均保持了更低的平均延迟。

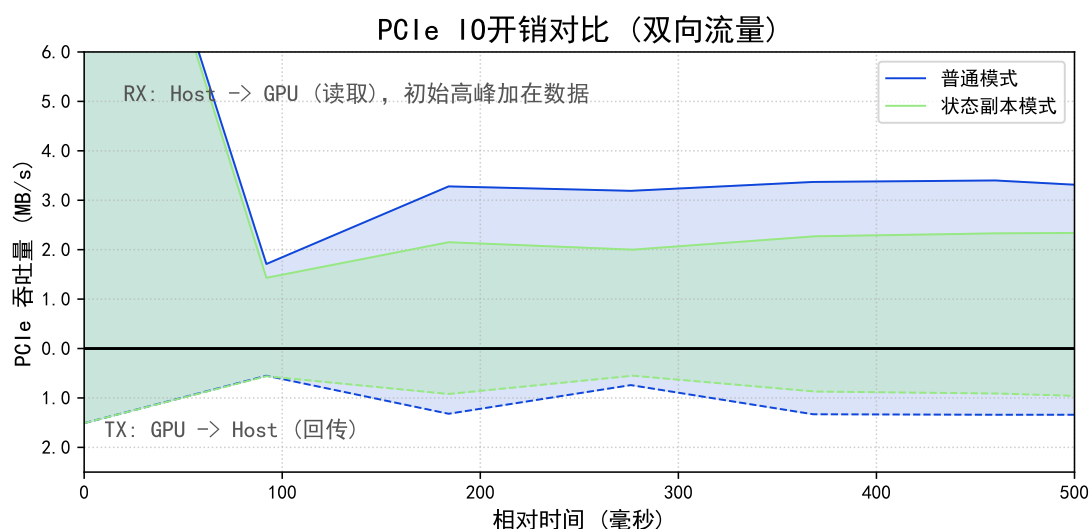


图 4.8 两种模式下 PCIe 实时 I/O 吞吐量监控对比 (RX 为主机发送, TX 为 GPU 回传)

为了进一步显示状态副本的有效性，我们在 GLoVe 数据集搜索上实时监控了 PCIe 总线上的流量负载，如图 4.8 所示。在普通模式下（蓝色实线），RX 端（主机读取 GPU 状态）和 TX 端（GPU 回传数据）均保持了较高的基线流量，这是因为主机需要不断向 GPU 发送 PCIe Read 请求以轮询查询是否完成，这种“远程轮询”占据了一定的带宽。相反，状态副本模式（绿色阴影区域）利用 GPU 的 DMA 引擎将完成状态主动写入 Host 端的锁页内存，主机仅需读取本地内存即可获知状态，从而消除了 PCIe 上的轮询流量。如图所示，该机制使 RX 端的平均带宽占用从约 3.2 MB/s 降低至 2.0 MB/s 左右。并且状态的读写是 PCIe 小包传输，降低 1MB 左右的小包传输带宽实际上会减少大量的 PCIe 事务。这种本地轮询策略能够减少 PCIe 事务的带宽，为核心的向量数据传输预留了更多的总线带宽，从而实现了整体性能的提升。

三、束搜索策略的有效性分析

为了验证分阶段束搜索在端到端检索任务中的实际收益，我们在不同数据集上对比了束搜索模式与传统贪婪搜索模式的吞吐量与延迟随召回率的变化关系。

如图 4.9 所示，我们在 GIST-1M、NYTimes 和 GloVe200 数据集上绘制了

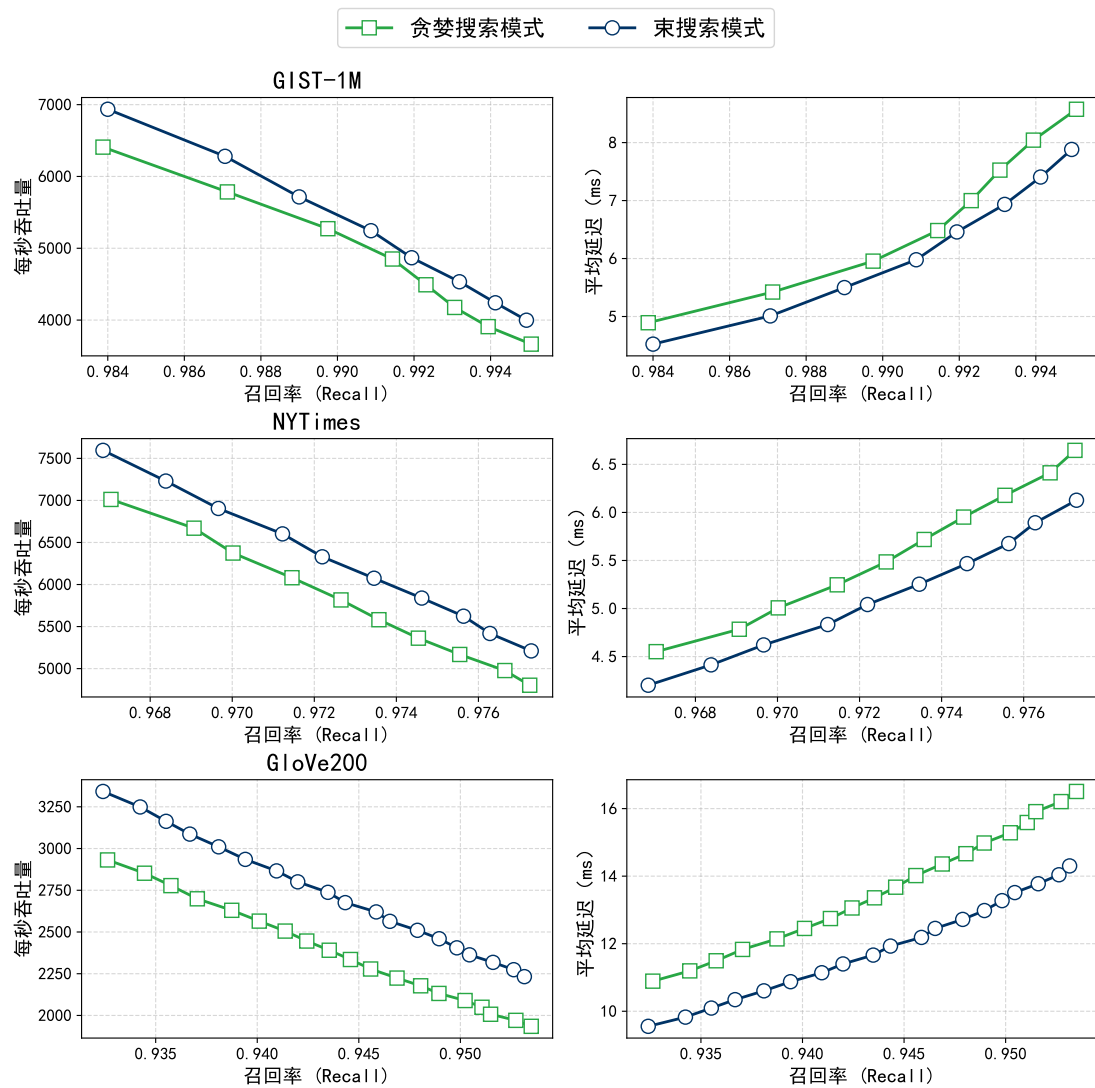


图 4.9 束搜索与贪婪搜索在不同数据集下的性能（吞吐量与延迟）随召回率变化的对比

性能曲线。吞吐量曲线越靠近右上方越优，延迟曲线越靠近右下方越优。因 SIFT-1M 数据集性能在贪婪搜索模式下已达饱和状态，图中未予展示。从实验结果可以清晰地观察到，束搜索模式（蓝色圆形曲线）在所有测试数据集中均展现出了比贪婪搜索模式（绿色方形曲线）更优的性能曲线。

在相同的目标召回率约束下，束搜索始终保持着更高的查询吞吐量。特别是在对召回率要求极高（如 $\text{Recall} > 0.98$ ）的区间内，贪婪搜索因频繁的单点回溯导致性能急剧下降，而束搜索凭借批量扩展机制，在维持高精度的同时有效遏制了吞吐量的下滑趋势。对应地在延迟表现上，束搜索在全召回率范围内均实现了更低的平均响应时间。

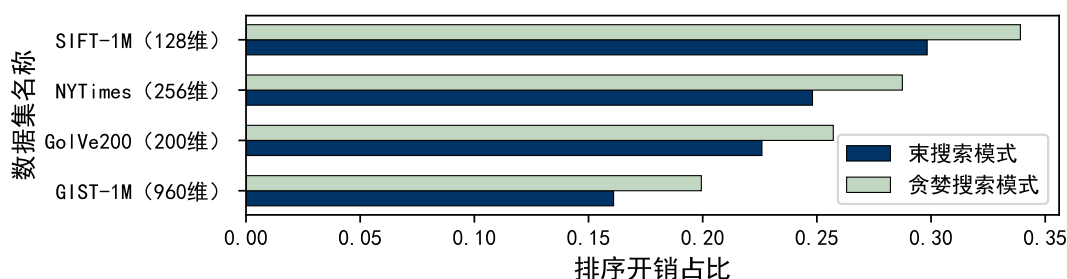


图 4.10 不同维度数据集下，束搜索与贪婪搜索中排序操作的时间开销占比对比

为了进一步探究束搜索带来性能提升和排序开销的影响，我们对搜索过程中的开销进行了分解测量，重点关注了列表排序与维护操作。图 4.10 展示了在四个不同维度的数据集（从 128 维的 SIFT-1M 到 960 维的 GIST-1M）上，排序操作在总查询延迟中的时间占比。

实验数据表明，与贪婪搜索相比，束搜索模式下排序操作的时间占比在所有数据集中均有显著下降。例如在 SIFT-1M 数据集中，排序开销占比从约 35% 降低至 30% 以下；在 GIST-1M 中也有明显回落。这验证了我们前文的理论分析：束搜索通过延迟更新，将多次细粒度的“单点排序与合并”聚合为少次高效的“批量排序与合并”，从而有效摊薄了维护候选列表的计算成本，提升了 GPU 指令流水线的执行效率。

四、本章小结

本章对本文提出的 ALGAS 系统进行了全面的实验评估与深入分析。基于 NVIDIA RTX A6000 硬件平台，我们使用了涵盖计算机视觉与自然语言处理领

域四个高维向量数据集（SIFT-1M, GIST-1M, GloVe200, NYTimes），从端到端性能、参数敏感性、核心模块有效性等多个维度进行了验证。

实验结果表明，ALGAS 在小批次综合性能上显著优于当前最先进的 GPU 图索引系统 CAGRA 与 GANNS。在相同的召回率水平下，ALGAS 实现了更高的吞吐量与更低的查询延迟，特别是在处理不同批次大小的负载时展现出了卓越的资源调度能力与鲁棒性。

此外，消融实验定量地验证了系统关键优化策略的有效性。首先，动态流式处理策略有效打破了传统批处理中的同步屏障，显著降低了长尾延迟并提升了系统响应速度；其次，多线程与状态副本机制通过优化 Host 端的并发调度与 PCIe 数据传输模式，大幅减少了总线上的控制信令开销，解除了高频小包传输下的 IO 瓶颈；最后，分阶段束搜索通过延迟更新策略有效摊薄了高昂的排序与列表维护开销，在保证高召回精度的同时，实现了比贪婪搜索更优的性能。

第五章 总结展望

第一节 本文工作总结

在大语言模型（LLM）与检索增强生成（RAG）技术爆发式增长的背景下，近似最近邻检索（ANNS）已成为现代人工智能基础设施与搜索推荐引擎的核心基石。图形处理器（GPU）凭借其海量并发计算能力，在加速 ANNS 领域备受关注。尽管现有研究显著提升了大规模检索的吞吐量，但在面向在线服务的低延迟场景中仍面临严峻挑战。

在调度层面，由于 ANNS 的近邻图检索具有显著的长尾特性，传统的 GPU 同步批处理机制在小批次场景下，处理较快的查询被迫等待长尾查询，导致批次间出现严重的调度气泡与资源闲置。此外，频繁的核函数启动与跨设备通信开销进一步加剧了系统的端到端延迟。在算法执行层面，GPU 的并行架构极大加速了距离计算开销，使得近邻图检索中关于候选队列与相关数据结构维护的开销占比突出，这类维护操作存在一定程度串行，限制了性能的进一步提升；此外，目前低延迟的 Multi-CTA 并行模式下引入了的各 CTA 分片结果合并的开销，这需要跨 SM 同步与全局显存的参与，抵消了并行计算带来的部分收益。

针对上述挑战，本文设计并实现了一种面向低延迟场景的高性能 GPU 向量检索系统，分别从调度架构与算法执行两个层面进行了设计与优化。在调度架构层面，本文提出了一种基于多槽位的动态流式处理机制。该机制摒弃了传统的同步 GPU 批处理屏障，将计算资源抽象为逻辑独立的执行槽位，允许查询任务异步进入与退出，从而彻底消除了长尾查询对整体批次的阻塞效应。为配合动态调度，系统设计了持久化核函数策略，通过一次性驻留 GPU 的轮询内核替代频繁的内核启动，实现了任务的零开销分发；同时，引入基于本地副本的 PCIe 状态同步技术，利用统一地址空间技术（UVM）将 Host 与 Device 端的槽位状态映射到对方地址空间，有效消除了高频轮询下的 PCIe 总线 I/O 瓶颈。在算法执行层面，本文设计了软硬协同的计算优化策略。首先，提出分阶段束搜索算法，根据查询过程中的距离收敛特征，自适应地在“定位阶段”与“收敛阶段”切换搜索策略，通过延迟列表维护显著降低了维护列表的执行开销。其

次，构建了 CPU-GPU 异构协同流水线，利用 GPU 专注搜索，CPU 异步合并的解耦设计，将逻辑复杂但计算量较小的 Top-K 归并任务卸载至 Host 端，在充分利用 CPU 空闲算力的同时，有效掩盖了结果回传的延迟。

基于 NVIDIA RTX A6000 硬件平台与 SIFT-1M、GIST-1M 等多个高维基准数据集的实验结果表明，本系统在保持高召回率的同时，显著突破了现有方案的性能瓶颈。与当前业界最先进的 GPU 图索引系统 CAGRA 相比，本系统在平均查询延迟上降低了 21.9% 至 35.4%，同时吞吐量提升了 27.8% 至 55.2%。消融实验进一步证实，调度架构与算法策略在不同负载下均表现出卓越的有效性，为构建兼具高吞吐与极速响应的下一代向量检索系统提供了有效的技术路径。

第二节 未来研究展望

在实际的工业级推荐系统与 RAG（检索增强生成）应用中，单纯的向量相似度检索往往无法满足业务需求，查询请求通常伴随着复杂的标量属性过滤条件（如“查找价格低于 100 元且颜色为红色的相似商品”）。目前的 GPU 近邻图算法大多仅支持纯向量搜索，若采用先搜索后过滤（Post-filtering）策略，极易因 Top-K 结果被过滤殆尽而导致召回率骤降；若采用边搜索边过滤（Pre-filtering）策略，被过滤节点会导致图的连通性被破坏，使得贪婪搜索路径中断，陷入局部最优。

未来的工作方向可以致力于设计 GPU 友好的混合过滤图索引，探索如何在 GPU 显存受限条件下，如何高效地将标量属性索引（如倒排链或位图）与近邻图拓扑深度融合。重点研究方向包括：

- 设计支持 GPU 并行执行的位图过滤算子或逻辑，在不破坏 GPU 访存连续性的前提下实现在边搜索边过滤策略下高效的邻居剪枝。
- 研究连通性保持机制，在大量节点被属性过滤屏蔽的情况下，如何设计联合属性的近邻图索引，通过动态增加辅助边或长程跳跃连接，维持图的导航能力。

当前主流的 GPU 图索引（包括 CAGRA 和本文复用的索引）大多基于静态图假设，即索引构建完成后不支持新增或修改，数据更新往往需要昂贵的全量重建。然而，在新闻推荐、实时风控及流式 RAG 场景下，数据分布随时间快速漂移，要求系统具备高频写入与实时可见的能力。

在 GPU 上实现高效的动态图更新面临着显存管理与并发冲突的双重挑战。

未来的工作可以探索支持高并发读写分离的 GPU 动态图架构：

- 借鉴 LSM-Tree 的思想，设计基于 GPU 显存的分层索引结构。利用小规模动态图（MemTable）缓冲实时写入流量，利用大规模的静态基准图承载历史数据，并通过后台异步合并线程实现数据的定期重组与垃圾回收。
- 针对 GPU 显存分配器在动态增删节点时的碎片化问题，设计专用的块式显存管理机制，以支持邻接表的动态扩容与高效访问。

现有的向量检索研究通常将上游嵌入模型视为黑盒，仅针对既定的向量分布设计索引结构。然而，索引的性能（召回率与检索速度）在很大程度上受制于向量空间的分布特性（如维度的相关性、簇的紧密度）。这种模型与索引的割裂导致了潜在的性能天花板。未来的研究可以尝试打破这一界限，探索嵌入模型与索引的联合设计与协同优化：

- 比如在近邻图层面，可以设计支持索引感知的嵌入模型训练目标。在模型训练阶段引入索引构建的损失函数，引导模型生成更适合近邻图导航的向量分布（例如降低图的固有维度，或增强簇的分离度），从而降低图搜索的跳数。
- 此外还可以结合属性或标签联合训练，在嵌入模型中支持增加额外属性，这样可以在过滤搜索过程中直接使用普通的搜索模式，省去了属性过滤的相关开销。

参考文献

- [1] BRIN S, PAGE L. The anatomy of a large-scale hypertextual web search engine[J]. Computer networks and ISDN systems, 1998, 30(1-7): 107-117.
- [2] HUANG P S, HE X, GAO J, et al. Learning deep structured semantic models for web search using clickthrough data[C]//Proceedings of the 22nd ACM international conference on Information & Knowledge Management. 2013: 2333-2338.
- [3] COVINGTON P, ADAMS J, SARGIN E. Deep neural networks for youtube recommendations[C]//Proceedings of the 10th ACM conference on recommender systems. 2016: 191-198.
- [4] GOLDBERG D, NICHOLS D, OKI B M, et al. Using collaborative filtering to weave an information tapestry[J]. Communications of the ACM, 1992, 35(12): 61-70.
- [5] BROWN T, MANN B, RYDER N, et al. Language models are few-shot learners[J]. Advances in neural information processing systems, 2020, 33: 1877-1901.
- [6] LEWIS P, PEREZ E, PIKTUS A, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks[J]. Advances in neural information processing systems, 2020, 33: 9459-9474.
- [7] INDYK P, MOTWANI R. Approximate nearest neighbors: towards removing the curse of dimensionality[C]//Proceedings of the thirtieth annual ACM symposium on Theory of computing. 1998: 604-613.
- [8] MALKOV Y, PONOMARENKO A, LOGVINOV A, et al. Approximate nearest neighbor algorithm based on navigable small world graphs[J]. Information Systems, 2014, 45: 61-68.
- [9] MALKOV Y A, YASHUNIN D A. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs[J]. IEEE transactions on pattern analysis and machine intelligence, 2018, 42(4): 824-836.
- [10] LI W, ZHANG Y, SUN Y, et al. Approximate nearest neighbor search on high dimensional data—experiments, analyses, and improvement[J]. IEEE Transactions on Knowledge and Data Engineering, 2019, 32(8): 1475-1488.
- [11] FU C, XIANG C, WANG C, et al. Fast approximate nearest neighbor search with the navigating spreading-out graph[J]. arXiv preprint arXiv:1707.00143, 2017.
- [12] JEGOU H, DOUZE M, SCHMID C. Product quantization for nearest neighbor search[J]. IEEE transactions on pattern analysis and machine intelligence, 2010, 33(1): 117-128.
- [13] GAO J, LONG C. High-dimensional approximate nearest neighbor search: with reliable and efficient distance comparison operations[J]. Proceedings of the ACM on Management of Data, 2023, 1(2): 1-27.
- [14] GAO J, LONG C. Rabbitq: Quantizing high-dimensional vectors with a theoretical error bound

- for approximate nearest neighbor search[J]. *Proceedings of the ACM on Management of Data*, 2024, 2(3): 1-27.
- [15] HASSANTABAR S, WANG Z, JHA N K. SCANN: Synthesis of compact and accurate neural networks[J]. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2021, 41(9): 3012-3025.
- [16] DATAR M, IMMORLICA N, INDYK P, et al. Locality-sensitive hashing scheme based on p-stable distributions[C]//*Proceedings of the twentieth annual symposium on Computational geometry*. 2004: 253-262.
- [17] BENTLEY J L. Multidimensional binary search trees used for associative searching[J]. *Communications of the ACM*, 1975, 18(9): 509-517.
- [18] MUJA M, LOWE D G. Fast approximate nearest neighbors with automatic algorithm configuration.[J]. *VISAPP* (1), 2009, 2(331-340): 2.
- [19] JOHNSON J, DOUZE M, JÉGOU H. Billion-scale similarity search with GPUs[J]. *IEEE Transactions on Big Data*, 2019, 7(3): 535-547.
- [20] WANG J, YI X, GUO R, et al. Milvus: A purpose-built vector data management system[C]//*Proceedings of the 2021 international conference on management of data*. 2021: 2614-2627.
- [21] MOMJIAN B. PostgreSQL: introduction and concepts: vol. 192[M]. Addison-Wesley New York, 2001.
- [22] NVIDIA Corporation. NVIDIA Corporation Official Website[EB/OL]. 2025. <https://www.nvidia.com/>.
- [23] ZHAO W, TAN S, LI P. Song: Approximate nearest neighbor search on gpu[C]//*2020 IEEE 36th International Conference on Data Engineering (ICDE)*. 2020: 1033-1044.
- [24] YU Y, WEN D, ZHANG Y, et al. GPU-accelerated proximity graph approximate nearest Neighbor search and construction[C]//*2022 IEEE 38th International Conference on Data Engineering (ICDE)*. 2022: 552-564.
- [25] GROH F, RUPPERT L, WIESCHOLLEK P, et al. Ggnn: Graph-based gpu nearest neighbor search[J]. *IEEE Transactions on Big Data*, 2022, 9(1): 267-279.
- [26] KHAN S, SINGH S, SIMHADRI H V, et al. BANG: Billion-Scale Approximate Nearest Neighbor Search using a Single GPU[J]. *arXiv preprint arXiv:2401.11324*, 2024.
- [27] OOTOMO H, NARUSE A, NOLET C, et al. Cagra: Highly parallel graph construction and approximate nearest neighbor search for gpus[C]//*2024 IEEE 40th International Conference on Data Engineering (ICDE)*. 2024: 4236-4247.
- [28] WANG M, XU X, YUE Q, et al. A comprehensive survey and experimental comparison of graph-based approximate nearest neighbor search[J]. *arXiv preprint arXiv:2101.12631*, 2021.
- [29] ZHOU W, LI H, TIAN Q. Scalar quantization for large scale image search[C]//*Proceedings of the 19th ACM International Conference on Multimedia*. 2012: 169-178.
- [30] JUANG B H, GRAY A. Multiple stage vector quantization for speech coding[J]. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 1982, 30(4): 520-531.

- [31] NICKOLLS J, BUCK I, GARLAND M, et al. Scalable parallel programming with CUDA [J]. Queue, 2008, 6(2): 40-53.
- [32] CHEN W, CHEN J, ZOU F, et al. Robustiq: A robust ann search method for billion-scale similarity search on gpus[C]//Proceedings of the 2019 on international conference on multimedia retrieval. 2019: 132-140.
- [33] LEE Y, CHOI H, MIN S, et al. Anna: Specialized architecture for approximate nearest neighbor search[C]//2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA). 2022: 169-183.
- [34] JAYARAM SUBRAMANYA S, DEVVRIT F, SIMHADRI H V, et al. Diskann: Fast accurate billion-point nearest neighbor search on a single node[J]. Advances in Neural Information Processing Systems, 2019, 32.
- [35] CHEN Q, ZHAO B, WANG H, et al. Spann: Highly-efficient billion-scale approximate nearest neighborhood search[J]. Advances in Neural Information Processing Systems, 2021, 34: 5199-5212.
- [36] ZENG S, ZHU Z, LIU J, et al. DF-GAS: a Distributed FPGA-as-a-Service Architecture towards Billion-Scale Graph-based Approximate Nearest Neighbor Search[C]//Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture. 2023: 283-296.
- [37] TIAN B, LIU H, DUAN Z, et al. Scalable Billion-point Approximate Nearest Neighbor Search Using {SmartSSDs}[C]//2024 USENIX Annual Technical Conference (USENIX ATC 24). 2024: 1135-1150.
- [38] JANG J, CHOI H, BAE H, et al. CXL-ANNS: Software-Hardware collaborative memory disaggregation and computation for Billion-Scale approximate nearest neighbor search[C]//2023 USENIX Annual Technical Conference (USENIX ATC 23). 2023: 585-600.
- [39] TIAN B, LIU H, TANG Y, et al. Fusionanns: An efficient cpu/gpu cooperative processing architecture for billion-scale approximate nearest neighbor search[J]. arXiv preprint arXiv:2409.16576, 2024.
- [40] BATCHER K E. Sorting networks and their applications[C]//Proceedings of the AFIPS Spring Joint Computer Conference. 1968: 307-314.
- [41] NVIDIA. RTX 6000 Ada Generation Graphics Card[Z]. <https://www.nvidia.com/en-us/design-visualization/rtx-6000/>. Accessed: October 4, 2024. 2024.
- [42] SIFT and GIST[DS/OL]. 2010. <http://corpus-texmex.irisa.fr/>.
- [43] PENNINGTON J, SOCHER R, MANNING C D. Glove: Global vectors for word representation[C]//Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP). 2014: 1532-1543.
- [44] AUMÜLLER M, BERNHARDSSON E, FAITHFULL A. ANN-Benchmarks: A benchmarking tool for approximate nearest neighbor algorithms[J]. Information Systems, 2020, 87: 101374.

致谢

感谢您使用本模板。

个人简历

基本信息

姓名：陈源辉
性别：男
出生日期：2000 年 10 月 29 日
邮箱：yhchen@nbjl.nankai.edu.cn
电话：18359715052
地址：天津市南开大学津南校区计算机学院
主页：<https://nbjl.nankai.edu.cn/2023/0716/c32996a516482/page.htm>

教育背景

2023 年 09 月 – 2026 年 06 月	南开大学	计算机技术	工学	硕士
2019 年 09 月 – 2023 年 06 月	武汉理工大学	地理信息科学	理学	学士

研究生期间发表论文

- **Yuanhui Chen**, Lixiao Cui, Zebin Yao, Hao Zhou, Gang Wang, Xiaoguang Liu, ALGAS: A Low-latency GPU-Accelerated Approximate Nearest Neighbor Search System, IPDPS 2025 (CCF-B 类会议 IPDPS)
- Hao Zhou, **Yuanhui Chen**, Wu Zeng, Lixiao Cui, Gang Wang and Xiaoguang Liu, GPComp: Using GPU and SSD-GPU Peer to Peer DMA to Accelerate LSM-Tree Compaction for Key-Value Store, in IEEE Transactions on Parallel and Distributed Systems, Vol.36, No.9, pp.1920-1936, 2025, doi: 10.1109/TPDS.2025.3586616 (CCF-A 类期刊 TPDS)
- Zhou Hao, **Yuanhui Chen**, Cui Lixiao, Wang Gang, Liu Xiaoguang, A GPU-accelerated compaction strategy for LSM-based key-value store system, MSST 2024 (CCF-B 类会议 MSST)

研究生期间其它成果

- 2023 年 第 11 届亚太区 RDMA 编程竞赛 二等奖
- 2023 年 全国大学生系统能力大赛 省一等奖